

# ÍNDICE COMPLETO DEL CURSO DE LINUX CON UBUNTU

---

## ■ MÓDULO 1 — Introducción a Linux y Ubuntu

- 1.1. ¿Qué es Linux? Kernel vs GNU
  - 1.2. Distribuciones Linux y diferencias entre ellas
  - 1.3. Características de Ubuntu Desktop y Ubuntu Server
  - 1.4. Instalación de Ubuntu en máquina virtual
  - 1.5. Estructura básica del sistema
  - 1.6. Primeros pasos tras instalar Ubuntu
  - 1.7. Actualización y mantenimiento básico
- 

## ■ MÓDULO 2 — La Terminal y comandos esenciales

- 2.1. ¿Qué es el shell? Bash
  - 2.2. Navegación por el sistema de archivos
  - 2.3. Comandos básicos: ls, cd, pwd, mkdir, rm, cp, mv
  - 2.4. Visualización de archivos: cat, less, tail, head
  - 2.5. Redirecciones y tuberías (>, >>, |)
  - 2.6. Búsquedas con grep, find y wc
  - 2.7. Ayuda del sistema: man y apropos
  - 2.8. Variables de entorno
- 

## ■ MÓDULO 3 — Sistema de archivos Linux

- 3.1. Jerarquía FHS
  - 3.2. Directorios esenciales: /etc, /var, /usr, /home...
  - 3.3. Sistemas de archivos: EXT4, XFS, BTRFS
  - 3.4. Montaje y desmontaje de dispositivos
  - 3.5. Gestión de particiones
  - 3.6. El archivo /etc/fstab
  - 3.7. Inodos y metadatos
- 

## ■ MÓDULO 4 — Usuarios, grupos y permisos

- 4.1. Crear y gestionar usuarios
- 4.2. Crear y gestionar grupos
- 4.3. Permisos básicos (r w x)
- 4.4. chmod, chown, chgrp
- 4.5. Permisos especiales: SUID, SGID y Sticky Bit

- 4.6. Control de acceso avanzado con ACLs
  - 4.7. Archivos `/etc/passwd` y `/etc/shadow`
- 

## ■ **MÓDULO 5 — Procesos y servicios en Linux**

- 5.1. Procesos: PID, PPID, estados
  - 5.2. Comandos `ps`, `top`, `htop`
  - 5.3. Señales: `kill`, `killall`, `killall`
  - 5.4. Prioridades: `nice` y `renice`
  - 5.5. Qué es `systemd`
  - 5.6. Gestionar servicios con `systemctl`
  - 5.7. Logs del sistema: `journalctl`
- 

## ■ **MÓDULO 6 — Red en Linux**

- 6.1. Comandos de red: `ip`, `ping`, `traceroute`, `ss`, `dig`
  - 6.2. Configuración de IP estática y DHCP
  - 6.3. Configurar Netplan
  - 6.4. Puertos, sockets y conexiones activas
  - 6.5. Servidor SSH
  - 6.6. Firewall UFW
  - 6.7. Diagnóstico de fallos de red
- 

## ■ **MÓDULO 7 — Gestión de paquetes y repositorios**

- 7.1. `apt`: `update`, `upgrade`, `install`, `remove`
  - 7.2. Paquetes `.deb`
  - 7.3. Repositorios APT y PPAs
  - 7.4. Snap packages
  - 7.5. Limpieza del sistema: `autoremove`, `autoclean`
- 

## ■ **MÓDULO 8 — Scripting en Bash**

- 8.1. Crear scripts y hacerlos ejecutables
- 8.2. Variables y argumentos
- 8.3. Condicionales (`if`)
- 8.4. Bucles (`for`, `while`)
- 8.5. Funciones
- 8.6. Automatización con `cron`
- 8.7. Scripts útiles para administración

---

## ■ MÓDULO 9 — Seguridad en Linux

- 9.1. Seguridad del sistema operativo
  - 9.2. UFW: configuración básica y avanzada
  - 9.3. Hardening SSH
  - 9.4. Fail2ban
  - 9.5. Gestión de permisos y auditoría
  - 9.6. Análisis de logs de seguridad
- 

## ■ MÓDULO 10 — Proyecto final de Administración Linux

- 10.1. Diseño del entorno
- 10.2. Configuración completa del servidor
- 10.3. Seguridad aplicada
- 10.4. Automatización
- 10.5. Documentación técnica
- 10.6. Presentación del proyecto

# ■ MÓDULO 1 — INTRODUCCIÓN A LINUX Y UBUNTU

---

## ■ 1.1. ¿Qué es Linux? Kernel vs GNU

Linux **no es un sistema operativo completo**: es *solo el kernel*.

El sistema operativo como tal se construye sumando:

- **Kernel Linux** → administra hardware, memoria, procesos
- **Herramientas GNU** → comandos, librerías, compiladores
- **Software adicional** → servidores, escritorio, paquetes

La combinación **GNU + Linux** es lo que forma los sistemas que usamos hoy (Ubuntu, Debian, etc.).

---

## ■ 1.2. Distribuciones Linux

Una *distribución* es la forma en la que una organización empaqueta:

- Kernel

- Programas básicos
- Repositorios
- Gestor de paquetes
- Escritorio (GNOME, KDE, XFCE...)

Principales familias:

- **Debian/Ubuntu** → fáciles, muy usadas en servidores y FP
- **Red Hat/CentOS/Rocky** → empresas, cloud, corporaciones
- **Arch Linux** → rolling release, avanzada
- **OpenSUSE** → YaST, entorno empresarial
- **Kali Linux** → ciberseguridad
- **Linux Mint** → usuarios principiantes

Para este curso usamos **Ubuntu LTS**, la más estable para formación.

---

### 1.3. Ubuntu Desktop vs Ubuntu Server

| Ubuntu Desktop          | Ubuntu Server              |
|-------------------------|----------------------------|
| Entorno gráfico (GNOME) | Sin interfaz gráfica       |
| Para usuarios finales   | Para administradores       |
| Apps instaladas         | Ligero y optimizado        |
| Se usa en aulas FP      | Se usa en empresas y cloud |

Trabajaremos **con ambos**, según el módulo.

---

### 1.4. Instalación de Ubuntu en máquina virtual

Se recomienda **VirtualBox** o **VMware** para evitar problemas en el equipo real.

Pasos:

1. Descargar Ubuntu LTS
2. Crear VM con 2–4GB RAM y 20–40GB disco
3. Montar ISO

4. Elegir idioma y teclado
  5. Particionado automático
  6. Crear usuario y contraseña
  7. Primer arranque + actualizaciones
- 

## 1.5. Estructura básica del sistema

Linux se organiza en un **árbol jerárquico** que parte de /.

Directorios importantes:

- /home → usuarios
- /etc → configuración
- /bin /sbin /usr/bin → comandos
- /var → logs y servicios
- /root → superusuario
- /boot → arranque
- /media /mnt → dispositivos externos

### Ejercicio:

Usando cd, intenta entrar en cada una de las carpetas

---

## 1.6. Primeros pasos tras instalar Ubuntu

Comandos esenciales:

- sudo apt update
- sudo apt upgrade

sudo apt install htop curl gitsud

htop; es una aplicación que analiza el tráfico de red

curl: me permite ver contenidos web en modo consola

Tareas recomendadas:

- Configurar idioma
- Instalar herramientas base

- Ver estado de red
  - Revisar versiones de kernel:
  - `uname -a`
- 

## 1.7. Actualización y mantenimiento básico

Ubuntu usa APT:

- `sudo apt update`
- `sudo apt upgrade`
- `sudo apt autoremove`
- `sudo apt clean`

Es fundamental mantener el sistema actualizado para evitar vulnerabilidades.

---

## 20 EJERCICIOS PRÁCTICOS — MÓDULO 1

A continuación tienes una batería de ejercicios para FP, desde nivel básico a intermedio.

---

### EJERCICIOS BÁSICOS (1–10)

#### 1. Instalar Ubuntu en una máquina virtual

- Crea una VM con 2GB RAM y 20GB disco.
  - Instala Ubuntu.
  - Arranca correctamente.
- 

#### 2. Identificar la versión del sistema

Ejecuta:

- `uname -a`

→ Anota versión de Ubuntu y kernel.

---

#### 3. Actualizar el sistema

Ejecuta:

- `sudo apt update`
- `sudo apt upgrade`

→ Indica cuántos paquetes se actualizaron.

---

#### 4. Localizar los principales directorios

Usa:

- ls /

→ Identifica /etc, /home, /var, /usr.

---

#### 5. Abrir terminal y ejecutar comandos básicos

Ejecuta:

- pwd
- whoami
- date
- uptime

→ Explica cada resultado.

En el caso de que salga /, significa que estas en el directorio raíz o root directory

---

#### 6. Identificar el shell por defecto

Ejecuta:

- echo \$SHELL

→ ¿Usas Bash o Zsh?

---

#### 7. Explorar dispositivos montados

Ejecuta:

- lsblk
- df -h

→ Identifica el disco virtual.

---

#### 8. Examinar los procesos iniciales

Ejecuta:

- ps -ef | head

→ Anota qué procesos se ejecutan al arrancar.

---

## 9. Instalar un programa

- Instalar htop:
- `sudo apt install htop`

→ Abrirlo y observar consumo.

---

## 10. Apagar y reiniciar desde terminal

- `sudo reboot`
  - `sudo poweroff`
- 

## ✓ EJERCICIOS INTERMEDIOS (11–20)

### 11. Crear un usuario nuevo

- `sudo adduser alumno`

→ Asignar contraseña.

Si queremos irnos a este usuario

- `Su alumno`

Me pedirá la contraseña

Pero si quiero volver en nuestro caso a osboxes

- `Su osboxes`

Volverá a pedir la contraseña

---

### 12. Explorar contenido del directorio /etc

`ls /etc | head -n 20`

→ Identifica 20 archivos de configuración.

---

### 13. Ver logs básicos del sistema

`sudo tail /var/log/syslog`

---



## 14. Comprobar red

- ip a
- ping 8.8.8.8 -c 4

→ Determina si tienes conexión.

---

## 15. Cambiar la resolución del escritorio (Ubuntu Desktop)

A través de Configuración → Pantalla.

---

## 16. Instalar las Guest Additions en VirtualBox

Para mejorar pantalla, carpetas compartidas y portapapeles.

En osboxes.org ya vienen instaladas

---

## 17. Investigar qué servicios están ejecutándose

```
systemctl --type=service --state=running
```

---

## 18. Ver cuánta RAM usa el sistema recién instalado

```
free -h
```

---

## 19. Ver el historial de comandos

```
history | tail -n 20
```

→ ¿Qué comando se ejecutó en el número 100?

## 20. ver los logs del sistema

**Uso básico de journalctl**

**Ver todos los logs**

```
journalctl
```

 Se abre con paginador (sales con q).

---

**Ver logs más recientes (últimas líneas)**

```
journalctl -n 50
```

---

### Ver logs en tiempo real (tipo tail -f)

journalctl -f

---

### Mostrar solo el arranque actual

journalctl -b

---

### Ver logs del arranque anterior

journalctl -b -1

---



### Filtrar por tiempo

#### Desde hoy

journalctl --since today

#### Desde una fecha

journalctl --since "2026-01-01"

#### Entre fechas

journalctl --since "2026-01-01 10:00" --until "2026-01-01 12:00"

---



### Filtrar por servicio (MUY IMPORTANTE)

Ejemplo SSH:

journalctl -u ssh

Ejemplo networking:

journalctl -u NetworkManager

Ver en tiempo real:

journalctl -u ssh -f

---



### Filtrar por nivel de error

#### Solo errores

```
journalctl -p err -b
```

### Desde warning hacia arriba

```
journalctl -p warning -b
```

Niveles: emerg alert crit err warning notice info debug

---

### Filtrar por usuario

Logs de un usuario (por UID):

```
journalctl _UID=1000
```

---

### Buscar texto dentro de los logs

Ejemplo buscar “fail”:

```
journalctl | grep -i fail
```

o directamente:

```
journalctl -b | grep -i error
```

---

### 5 comandos imprescindibles (chuleta)

```
journalctl -b          # logs del arranque actual
```

```
journalctl -u ssh -n 50  # últimas 50 líneas del servicio ssh
```

```
journalctl -f          # logs en tiempo real
```

```
journalctl -p err -b    # errores del arranque actual
```

```
journalctl --since today # logs desde hoy
```

---

## 21. Crear un documento técnico

Redacta un archivo `introduccion_linux.txt` con:

- Qué es Linux
- Qué es Ubuntu
- Qué beneficios tiene

- Qué has aprendido en el Módulo 1

Guárdalo en /home/alumno.

---

## MÓDULO 2 — LA TERMINAL Y COMANDOS ESENCIALES

---

### 2.1. ¿Qué es el shell? ¿Qué es Bash?

El **shell** es la interfaz que permite al usuario escribir comandos para comunicarse con el sistema operativo.

Existen varios:

- **Bash** → El más común en Linux (Ubuntu, Debian)
- **Zsh** → Muy popular en macOS y avanzados
- **Fish** → Interactivo
- **Dash** → Rápido, usado en scripts del sistema

Ubuntu usa **Bash** de forma predeterminada.

Puedes comprobarlo con:

```
echo $SHELL
```

---

### 2.2. Navegación por el sistema de archivos

Comandos principales:

| Comando | Función                    |
|---------|----------------------------|
| pwd     | Muestra ruta actual        |
| cd      | Cambiar de directorio      |
| ls      | Listar archivos y carpetas |
| ls -l   | Listado detallado          |
| ls -a   | Incluye archivos ocultos   |

## Comando Función

|      |                                |
|------|--------------------------------|
| tree | Vista en árbol (si se instala) |
|------|--------------------------------|

Ejemplo:

cd /etc

ls -l

pwd

---

## 2.3. Creación y manipulación de archivos

### Archivos

touch archivo.txt    # Crear archivo vacío

cp origen destino    # Copiar archivo

mv origen destino    # Mover o renombrar

rm archivo.txt        # Eliminar archivo

### Directorios

mkdir carpeta

mkdir -p ruta/completa

rm -r carpeta

---

## 2.4. Visualización de contenido

### Comando Uso

|     |                      |
|-----|----------------------|
| cat | Ver archivo completo |
|-----|----------------------|

|      |                         |
|------|-------------------------|
| less | Ver archivo por páginas |
|------|-------------------------|

|      |                 |
|------|-----------------|
| head | Primeras líneas |
|------|-----------------|

|      |                |
|------|----------------|
| tail | Últimas líneas |
|------|----------------|

|         |                                   |
|---------|-----------------------------------|
| tail -f | Ver archivo en tiempo real (logs) |
|---------|-----------------------------------|

Ejemplo:

```
tail -f /var/log/syslog
```

---

## 2.5. Redirecciones y tuberías

| Operador | Función                   |
|----------|---------------------------|
| >        | Sobrescribe archivo       |
| >>       | Añade al final            |
| ,        | ,                         |
| <        | Usar archivo como entrada |

Ejemplos:

```
ls -l > listado.txt
```

```
echo "Hola" >> saludo.txt
```

```
cat archivo.txt | grep "error"
```

---

## 2.6. Búsquedas con grep, find y wc

### **grep – Buscar texto en archivos**

```
grep "error" syslog
```

```
grep -i "ubuntu" archivo.txt
```

### **find – Buscar archivos por nombre, tamaño, fecha**

```
find / -name "*.conf"
```

```
find /home -type f -size +10M
```

### **wc – Contar palabras, líneas, caracteres**

```
wc -l archivo.txt
```

---

## 2.7. Ayuda del sistema: man y apropos

```
man ls
```

```
man grep
```

```
apropos network
```

---

## 2.8. Variables de entorno

Variables como:

HOME

PATH

USER

PWD

Verlas:

echo \$PATH

env

Crear una temporal:

export SALUDO="Hola Linux"

echo \$SALUDO

---

## 30 EJERCICIOS PRÁCTICOS DEL MÓDULO 2

A continuación tienes una serie de ejercicios progresivos y perfectos para FP.

---

### EJERCICIOS BÁSICOS (1–10)

#### 1. Mostrar el directorio actual

Ejecuta:

pwd

Anota la ruta.

---

#### 2. Crear una carpeta llamada “practicas” en tu home

mkdir ~/practicas

---

#### 3. Crear dentro de esa carpeta un archivo vacío

touch ~/practicas/ejemplo.txt

---

#### 4. Listar todos los archivos incluyendo ocultos

`ls -la`

---

#### 5. Moverte al directorio /etc y volver a tu home

`cd /etc`

`cd ~`

---

#### 6. Mostrar las primeras 10 líneas de un archivo

`head /etc/passwd`

---

#### 7. Mostrar las últimas 15 líneas de un archivo

`tail -n 15 /var/log/syslog`

---

#### 8. Ver el contenido del archivo passwd paginado

`less /etc/passwd`

---

#### 9. Crear un archivo con texto

`echo "Hola Linux" > mensaje.txt`

---

#### 10. Añadir una línea más al archivo

`echo "Bienvenido al curso" >> mensaje.txt`

---

### EJERCICIOS INTERMEDIOS (11–20)

#### 11. Copiar el archivo passwd a tu carpeta

`cp /etc/passwd ~/practicas/passwd_copia`

---

#### 12. Mover un archivo a otra ruta



`mv mensaje.txt ~/practicas/`

---

**13. Buscar la palabra “bash” dentro de passwd**

`grep "bash" /etc/passwd`

---

**14. Contar cuántas líneas tiene passwd**

`wc -l /etc/passwd`

---

**15. Crear 10 archivos automáticamente**

`touch archivo{1..10}.txt`

---

**16. Eliminar todos los archivos .txt**

`rm *.txt`

---

**17. Buscar archivos .conf en todo /etc**

`find /etc -name "*.conf"`

---

**18. Mostrar solo usuarios con /bin/bash**

`grep "/bin/bash" /etc/passwd`

---

**19. Guardar la salida de un comando en un archivo**

`ls -l /etc > listado.txt`

---

**20. Instalar el comando tree y ejecutarlo**

`sudo apt install tree`

`tree /`

---

 **EJERCICIOS AVANZADOS (21–30)**

## **21. Contar usuarios del sistema**

```
wc -l /etc/passwd
```

---

## **22. Mostrar procesos del usuario actual**

```
ps -u $USER
```

---

## **23. Buscar archivos mayores de 50 MB en el sistema**

```
find / -size +50M 2>/dev/null
```

---

## **24. Redirigir errores a un archivo**

```
ls /carpeta_que_no_existe 2> errores.txt
```

---

## **25. Concatenar dos archivos en uno**

```
cat archivo1 archivo2 > combinado.txt
```

---

## **26. Mostrar variables de entorno**

```
env
```

---

## **27. Añadir una variable al entorno**

```
export CURSO="Linux FP"
```

```
echo $CURSO
```

---

## **28. Buscar comandos relacionados con red**

```
apropos network
```

---

## **29. Mostrar un archivo en formato columna**

```
cat /etc/passwd | cut -d: -f1
```

---

### 30. Crear un pequeño script usando comandos del módulo

Contenido del script:

- `#!/bin/bash`
- `echo "Usuario: $USER"`
- `echo "Directorio: $PWD"`
- `date`

Haces ejecutable:

- `chmod +x info.sh`
- `./info.sh`

---

## MÓDULO 3 — SISTEMA DE ARCHIVOS EN LINUX

---

### 3.1. La jerarquía FHS (Filesystem Hierarchy Standard)

Linux no divide sus unidades en “C:”, “D:”, etc., sino que usa un **árbol único** cuyo punto inicial es `/`.

Las rutas se organizan así:

`/`

- └─ `bin` → Ejecutables básicos
- └─ `boot` → Kernel, GRUB...
- └─ `dev` → Dispositivos (discos, USB...)
- └─ `etc` → Configuración del sistema
- └─ `home` → Carpetas de usuarios
- └─ `lib` → Bibliotecas del sistema
- └─ `media` → Montaje automático de USB/externos
- └─ `mnt` → Montaje manual de dispositivos
- └─ `opt` → Programas externos
- └─ `root` → Home del usuario root
- └─ `sbin` → Ejecutables del sistema
- └─ `tmp` → Archivos temporales

└─ usr → Programas instalados  
└─ var → Datos variables (logs, colas, BD...)

---

## ■ 3.2. Directorios esenciales y su función

### /etc

Contiene **toda la configuración del sistema**:

- passwd, shadow → usuarios
- fstab → montaje
- hosts, hostname → red
- Configs de servicios (ssh, apache...)

### /var

Datos que cambian continuamente:

- /var/log → logs
- /var/www → webs
- /var/mail → correo

### /home

Almacena los perfiles y archivos de cada usuario.

### /dev

Representa **dispositivos** como si fueran archivos.

Ejemplo:

- /dev/sda → primer disco
- /dev/sda1 → primera partición

### /mnt y /media

- /mnt → montajes manuales o temporales
  - /media → montajes automáticos de USB, DVD, etc.
- 

## ■ 3.3. Sistemas de archivos en Linux

Los sistemas más comunes:

| Sistema     | Características               |
|-------------|-------------------------------|
| EXT4        | El más usado, robusto, rápido |
| XFS         | Ideal para grandes volúmenes  |
| BTRFS       | Snapshots, compresión, RAID   |
| NTFS        | Compatible con Windows        |
| FAT32/exFAT | USB, compatibilidad universal |

---

### 3.4. Montaje y desmontaje de dispositivos

Comandos clave:

- `lsblk`      # Ver discos y particiones
- **`sudo mount /dev/sdb1 /mnt`**
- **`sudo umount /mnt`**

Para ver dispositivos USB:

- `dmesg | tail`
- 

### 3.5. Gestión de particiones

Herramientas disponibles:

- **`fdisk` → particiones MBR**
- **`gdisk` → particiones GPT**
- **`parted` → general**
- **`gparted` → interfaz gráfica**

Ejemplo:

```
sudo fdisk /dev/sdb
```

---

### 3.6. Montaje automático con `/etc/fstab`

El archivo `/etc/fstab` indica qué se monta al inicio.

Formato típico:

```
UUID=xxxxxx /datos ext4 defaults 0 2
```

Consultar UUIDs:

```
blkid
```

---

### 3.7. Inodos y metadatos

Un **inodo** almacena:

- Permisos
- Propietario
- Tamaño
- Bloques ocupados
- Fechas (creación, acceso, modificación)

Comprobar inodo:

```
ls -li archivo.txt
```

```
stat archivo.txt
```

---

## 30 EJERCICIOS PRÁCTICOS — MÓDULO 3

Divididos en básicos, intermedios, avanzados y un mini-proyecto final.

---

### EJERCICIOS BÁSICOS (1–10)

#### 1. Mostrar la estructura del sistema

```
ls /
```

#### 2. Ver contenido de /etc

```
ls -l /etc
```

#### 3. Listar todos los dispositivos de bloque

```
lsblk
```

#### 4. Mostrar los inodos de varios archivos

```
ls -li /etc/passwd
```

```
ls -i /var/log/syslog
```

#### **5. Ver información completa de un archivo**

```
stat /etc/hosts
```

#### **6. Localizar dónde está montado un USB**

```
df -h
```

#### **7. Crear un directorio para montajes manuales**

```
sudo mkdir /mnt/usbtest
```

#### **8. Cambiar permisos a un directorio**

```
sudo chmod 755 /mnt/usbtest
```

#### **9. Identificar tu sistema de archivos principal**

```
df -T /
```

#### **10. Ver logs recientes del sistema**

```
sudo tail /var/log/syslog
```

---

### **✓ EJERCICIOS INTERMEDIOS (11–20)**

#### **11. Montar un dispositivo USB manualmente**

```
sudo mount /dev/sdb1 /mnt/usbtest
```

#### **12. Desmontarlo**

```
sudo umount /mnt/usbtest
```

#### **13. Ver UUID de todas las particiones**

```
blkid
```

#### **14. Añadir una entrada de prueba en /etc/fstab (comentada)**

Añade una línea:

```
# /dev/sdb1 /mnt/usbtest ext4 defaults 0 2
```

#### **15. Crear un archivo grande de forma artificial**

```
dd if=/dev/zero of=archivo_grande.img bs=1M count=100
```

#### **16. Ver cuánto espacio ocupa un directorio**

```
du -sh /var
```

### 17. Ver espacio libre del disco

```
df -h
```

### 18. Buscar archivos mayores de 100MB

```
find / -size +100M 2>/dev/null
```

### 19. Mostrar solo directorios dentro de /home

```
ls -d /home/*/
```

### 20. Ver qué servicios escriben en /var/log

```
ls /var/log
```

---

## EJERCICIOS AVANZADOS (21–30)

### 21. Crear una partición nueva en un disco secundario

*(usar VM para no dañar sistema)*

```
sudo fdisk /dev/sdb
```

---

### 22. Formatear la partición recién creada

```
sudo mkfs.ext4 /dev/sdb1
```

---

### 23. Montarla automáticamente en /etc/fstab

Usa UUID para evitar errores.

---

### 24. Comprobar si un disco tiene errores

```
sudo fsck /dev/sdb1
```

---

### 25. Ver número de inodos libres

```
df -i
```

---

### 26. Crear un archivo de 1GB usando fallocate

```
fallocate -l 1G archivo1G.img
```



---

## 27. Ver diferencias de tamaño entre bloque lógico y físico

```
sudo blockdev --getbsz /dev/sda
```

---

## 28. Analizar metadata de un archivo comprimido

```
stat archivo.tar.gz
```

---

## 29. Comprobar el tipo exacto de sistema de archivos

```
sudo blkid /dev/sda1
```

---

## 30. Ver el árbol completo de un directorio

Instalar tree:

```
sudo apt install tree
```

```
tree /etc
```

---

## MINI-PROYECTO DEL MÓDULO 3

Realiza la siguiente configuración completa:

### Objetivo:

Crear un segundo disco virtual, particionarlo, formatearlo, montarlo y configurarlo para que Ubuntu lo monte automáticamente al iniciar.

### Pasos a realizar:

1. Añadir un disco de 2GB a la VM
2. Detectarlo:
3. lsblk
4. Particionarlo:
5. `sudo fdisk /dev/sdb`
6. Formatearlo a EXT4:
7. `sudo mkfs.ext4 /dev/sdb1`
8. Crear punto de montaje:

9. `sudo mkdir /datos`
10. Montarlo:
11. `sudo mount /dev/sdb1 /datos`
12. Obtener UUID:
13. `blkid`
14. Añadir a `/etc/fstab`:
15. `UUID=xxxxx /datos ext4 defaults 0 2`
16. Reiniciar sistema
17. Verificar montaje automático:

`df -h`

---

## MÓDULO 4 — USUARIOS, GRUPOS Y PERMISOS EN LINUX

---

### 4.1. Conceptos básicos: usuarios, grupos y UID/GID

Linux es un sistema **multiusuario**.

Cada usuario tiene:

- **UID**: Identificador numérico
- **GID**: Grupo principal
- **Grupos secundarios**
- **Home directory**
- **Shell asignado**

Consulta tu usuario:

`id`

Tipos de usuarios:

- **root** → administrador total
- **usuarios normales**

- **usuarios del sistema** (servicios como www-data)
- 

## 4.2. Crear y gestionar usuarios

Comando recomendado en Ubuntu:

1. `sudo adduser nombre`

Para modificar usuario:

- `sudo usermod -aG grupo usuario`
- `sudo usermod -s /bin/bash usuario`
- `sudo usermod -d /nueva/ruta usuario`

Para eliminar:

- `sudo deluser usuario`
  - `sudo deluser --remove-home usuario`
- 

## 4.3. Crear y gestionar grupos

Crear grupos:

- `sudo groupadd docentes`
- `sudo groupadd alumnos`

Añadir usuario a un grupo:

- `sudo usermod -aG docentes juan`

Listar grupos:

- `groups juan`
- `getent group`

**Ver usuarios**

- `Cat /etc/passwd`

**Ver solo los nombres de usuarios**

- `Cut -d: -f1 /etc/passwd`

**Buscar un usuario**

- `Getent passwd juan`

**Ver los grupos**

- `Cat /etc/group`

### Ver solo nombres de grupos

- Cut -d: -f1 /etc/group

### Buscar un grupo concreto

- Getent group alumnos

### Ver a que grupo pertenece un usuario

- Groups juan o también id juan

### Ver usuarios de un grupo concreto

- Getent group alumnos

### Borrar usuario y grupos

1º borrar usuario pero dejar su carpeta

- Sudo deluser juan

2º borrar usuario y carpetas

- Sudo deluser --remove-home juan

3º borrar un grupo

- Sudo groupdel alumnos

---

## 4.4. Permisos básicos (r, w, x)

Cada archivo tiene permisos para:

- **Propietario (u)**
- **Grupo (g)**
- **Otros (o)**

Las letras significan:

- r → leer
- w → escribir
- x → ejecutar

Ejemplo de ls -l:

-rwxr-x---

---

## 4.5. Cambiar permisos y propietarios

Cambiar permisos:

- `chmod 755 archivo`
- `chmod u+r,g-w,o-x archivo`

el permiso 777

Cada número es la suma de permisos:

- **r** = 4 → leer (read)
- **w** = 2 → escribir (write)
- **x** = 1 → ejecutar / entrar (execute)

Entonces:

✓ **7 = 4+2+1 = rwx**

(lectura + escritura + ejecución)

---

### ✓ ¿Qué significa 777 exactamente?

`chmod 777 archivo_o_carpeta`

→ permisos:

- **Dueño:** rwx
- **Grupo:** rwx
- **Otros:** rwx

📌 Todo el mundo puede:

- ✓ leer
- ✓ modificar
- ✓ borrar
- ✓ ejecutar/entrar

| Nº | Permis | Qué permite     |
|----|--------|-----------------|
| 0  | ---    | nada            |
| 1  | --x    | ejecutar/entrar |

|   |     |                            |
|---|-----|----------------------------|
| 2 | -w- | escribir (sin leer)        |
| 3 | -wx | escribir + ejecutar        |
| 4 | r-- | leer                       |
| 5 | r-x | leer + ejecutar            |
| 6 | rw- | leer + escribir            |
| 7 | rwX | leer + escribir + ejecutar |

Si por ejemplo usando el usuario principal parrot usamos

`Ls -ld /home/parrot`

Perfecto 👍 Ese resultado significa:

- **d** → es un directorio
- **rwXr-Xr-X** → permisos **755**
- Propietario: **parrot**
- Grupo: **parrot**
- Carpeta: **/home/parrot**

✅ O sea: **tu usuario parrot es dueño del directorio**, así que puedes cambiar permisos **sin sudo**.

La explicación es:

### 1) Primer bloque: **rwX** (propietario)

El dueño puede:

- **r** leer ✅ **w** escribir ✅ **X** entrar/ejecutar ✅

➡ Resultado: **control total del propietario**

---

### 2) Segundo bloque: **r-X** (grupo)

Los usuarios del mismo grupo pueden:

- **r** leer ✅ **w** escribir ❌ **X** entrar/ejecutar ✅

➡ Resultado: **pueden ver y entrar, pero no modificar**

---

### 3) Tercer bloque: r-x (otros)

Cualquier otro usuario del sistema puede:

- r leer  w escribir  x entrar/ejecutar 

➡ Resultado: **pueden ver y entrar, pero no modificar**

### 4) Tabla de combinaciones más usadas (MUY práctica)

| Permis | Archivos  | Carpetas  | Uso típico                              |
|--------|-----------|-----------|-----------------------------------------|
| o      |           |           |                                         |
| 600    | rw-----   | rwX-----  | archivo privado (contraseñas)           |
| 644    | rw-r--r-- | rwXr-Xr-X | archivo normal público lectura          |
| 700    | rwX-----  | rwX-----  | carpeta privada usuario                 |
| 750    | rwXr-X--- | rwXr-X--- | carpeta compartida con grupo            |
| 755    | rwXr-Xr-X | rwXr-Xr-X | scripts / programas / carpetas públicas |
| 770    | rwXrwx--- | rwXrwx--- | carpeta de grupo (equipo)               |
| 775    | rwXrwxr-X | rwXrwxr-X | carpeta compartida, lectura para otros  |
| 777    | rwXrwxrwX | rwXrwxrwX | <b>todo el mundo</b> (inseguro)         |

### NOTA

**virgulilla** (el símbolo ~), que es como se llama oficialmente al rabito de la "ñ". En el mundo de Linux y Unix es un símbolo fundamental. Para que aparezca usamos ALTGR +4

Cambiar propietario:

- sudo chown usuario archivo
- sudo chown usuario:grupo archivo

si quiero ver los permisos de un archivo

ls -l archivo.txt

## 4.6. Permisos especiales: SUID, SGID, Sticky Bit

### SUID

El programa se ejecuta con permisos del propietario.

Ejemplo:

- `chmod u+s archivo`

### SGID

Archivos creados en ese directorio pertenecen al grupo.

- `chmod g+s directorio`

### Sticky Bit

En directorios compartidos solo el dueño puede borrar.

- `chmod +t /compartida`

---

## 4.7. Control de acceso avanzado con ACLs

Permite permisos más detallados:

Activar ACL:

- `sudo apt install acl`

Ver ACL:

- `getfacl archivo`

Asignar ACL:

- `setfacl -m u:alumno:rwX archivo`

---

## 4.8. Archivos relevantes del sistema

### **/etc/passwd**

Guarda usuarios.

### **/etc/shadow**

Contraseñas cifradas.



**/etc/group**

Grupos del sistema.

---

## **30 EJERCICIOS PRÁCTICOS DEL MÓDULO 4**

---

### **EJERCICIOS BÁSICOS (1–10)**

#### **1. Crear un usuario llamado alumno1**

```
sudo adduser alumno1
```

---

#### **2. Crear un grupo llamado profesores**

```
sudo groupadd profesores
```

---

#### **3. Añadir alumno1 al grupo profesores**

```
sudo usermod -aG profesores alumno1
```

---

#### **4. Mostrar la información del usuario**

```
id alumno1
```

---

#### **5. Crear un archivo y comprobar sus permisos**

```
touch prueba.txt
```

```
ls -l prueba.txt
```

---

#### **6. Cambiar permisos del archivo prueba.txt a 644**

```
chmod 644 prueba.txt
```

---

#### **7. Crear un directorio llamado compartida**

```
mkdir compartida
```

---

## 8. Poner permisos 755 al directorio compartida

```
chmod 755 compartida
```

---

## 9. Cambiar propietario del archivo prueba.txt a alumno1

```
sudo chown alumno1 prueba.txt
```

Si da error, podéis entrar con el usuario su parrot (por ejemplo) y ahí poneis la instrucción

## Nota

Si queremos que desde otro usuario como por ejemplo alumno1, podamos

utilizar sudo, usaríamos desde parrot en nuestro ejemplo, sudo usermod -aG sudo superalbertron

---

## 10. Cambiar grupo del archivo prueba.txt a profesores

```
sudo chown :profesores prueba.txt
```

---

## ✓ EJERCICIOS INTERMEDIOS (11–20)

### 11. Crear usuarios alumno2 y alumno3

```
sudo adduser alumno2
```

```
sudo adduser alumno3
```

---

### 12. Crear un grupo alumnos

```
sudo groupadd alumnos
```

---

### 13. Añadir los tres alumnos al grupo alumnos

```
sudo usermod -aG alumnos alumno1
```

```
sudo usermod -aG alumnos alumno2
```

```
sudo usermod -aG alumnos alumno3
```

---

#### **14. Crear carpeta /home/grupoalumnos**

```
sudo mkdir /home/grupoalumnos
```

---

#### **15. Asignar permisos para que solo el grupo alumnos pueda acceder**

```
sudo chown root:alumnos /home/grupoalumnos
```

```
sudo chmod 770 /home/grupoalumnos
```

---

#### **16. Aplicar SGID a la carpeta para que todo se cree con el grupo alumnos**

```
sudo chmod g+s /home/grupoalumnos
```

---

#### **17. Aplicar sticky bit a un directorio público**

```
sudo chmod +t /tmp
```

(También sirve crear uno propio)

---

#### **18. Listar solo los archivos que tengan sticky bit**

```
find / -type d -perm -1000 2>/dev/null
```

---

#### **19. Crear un archivo e implementar ACL**

```
touch archivo_acl
```

```
setfacl -m u:alumno2:rwX archivo_acl
```

---

#### **20. Quitar ACL al archivo**

```
setfacl -b archivo_acl
```

---

## ✓ EJERCICIOS AVANZADOS (21–30)

### 21. Crear estructura de usuarios:

- docentes
- administracion
- mantenimiento

Y asignar usuarios a cada grupo.

---

### 22. Crear carpeta departamental por grupo

Ejemplo:

/departamento\_docentes

/departamento\_admin

/departamento\_mantenimiento

---

### 23. Permitir acceso total al grupo y solo lectura a otros grupos

Usar combinación de:

chmod

chown

setfacl

---

### 24. Crear batch script que muestre todos los usuarios

Usa:

cut -d: -f1 /etc/passwd

---

### 25. Crear usuario sin acceso a shell

sudo usermod -s /usr/sbin/nologin alumnoN

---

### 26. Crear usuario con fecha de caducidad

sudo chage -E 2024-12-31 alumno1

---

### 27. Bloquear la cuenta de un usuario

```
sudo passwd -l alumno2
```

---

### 28. Desbloquear la cuenta

```
sudo passwd -u alumno2
```

---

### 29. Ver qué usuarios tienen shell interactivo

```
grep -E "bash|zsh" /etc/passwd
```

---

### 30. Detectar archivos cuyo propietario no existe

```
find / -nouser 2>/dev/null
```

---

## MINI-PROYECTO DEL MÓDULO 4

### Objetivo:

Configurar **un sistema multiusuario completo y seguro** para un aula FP.

### Tareas a realizar:

1. Crear grupos:
  - alumnos
  - profesores
  - administracion
2. Crear 10 usuarios:
  - 6 alumnos
  - 3 profesores
  - 1 administración
3. Crear directorios:
  - /aula/alumnos
  - /aula/profesores
  - /aula/comun

4. Asignar permisos:
    - alumnos → rwx en su carpeta / lectura en comun
    - profesores → rwx en su carpeta, lectura/escritura en comun
    - administracion → acceso completo a todo
  5. Aplicar SGID en carpetas departamentales
  6. Aplicar sticky bit en carpeta común
  7. Implementar ACL para dar permisos específicos a un usuario concreto
  8. Documentar toda la configuración
    - usuarios creados
    - grupos
    - permisos
    - comandos utilizados
- 

#### Nota

Si queremos introducir varios usuarios al mismo tiempo a un grupo:

```
for u in usuario1 usuario2 usuario3; do sudo usermod -aG nombre_del_grupo $u; done
```

## MÓDULO 5 — PROCESOS Y SERVICIOS EN LINUX

### 5.1. ¿Qué es un proceso en Linux?

Un **proceso** es un programa que se está ejecutando.

Cada proceso tiene:

- **PID** → Process ID
- **PPID** → Parent Process ID
- **Estado** → running, sleeping, zombie...
- **Consumo** → CPU, memoria, IO
- **Usuario propietario**
- **Señales** que puede recibir

Comandos básicos:

ps

top

htop

pgrep

pidof

---

## ■ 5.2. Jerarquía de procesos

Linux tiene una jerarquía:

systemd (PID 1)

├── sshd

├── NetworkManager

├── gdm

| └── gnome-shell

└── ...

Puedes verla con:

ps tree

---

## ■ 5.3. Comandos ps, top, htop

**ps**

Uso estándar:

- ps aux (procesos de todos los usuarios, pero solo terminal activo)
- ps -ef (muestra todos los procesos de todos los usuarios y terminales)

**top**

Monitor en tiempo real:

- k → matar proceso
- u → filtrar por usuario

- M → ordenar por RAM
- P → ordenar por CPU

## htop

Interfaz avanzada (instalar con apt):

- sudo apt install htop
- 

## 5.4. Señales: kill, pkill, killall

Enviar señal de parada:

- kill -9 PID # Forzar paro
- kill -15 PID # Paro elegante

Buscar por nombre:

2. pkill firefox
3. killall Firefox

## Nota

---

Si queremos saber el id de un proceso como Firefox

pgrep -a Firefox

En nuestro ejemplo daba el 4066

---

## 5.5. Prioridades: nice y renice

Un proceso con menos prioridad deja más CPU a otros.

Ejecutar proceso con prioridad baja:

- nice -n 10 comando

Ejemplo: nice -n 10 firefox

Cambiar prioridad mientras está en ejecución:

- sudo renice 5 -p PID
- 

## 5.6. ¿Qué es systemd?



Es el sistema de arranque de Ubuntu.

Gestiona:

- Servicios
- Dependencias
- Logs
- Arranque del sistema

Comandos:

- `systemctl start servicio`
- `systemctl stop servicio`
- `systemctl status servicio`
- `systemctl enable servicio`
- `systemctl disable servicio`

Ejemplo: Vamos a instalar el servicio apache2

- `sudo apt update`
- `sudo apt install apache2 -y`

Ejemplo instalar mariadb

- `sudo apt update`
- `sudo apt install mariadb-server mariadb-client -y`

## 5.7. Logs del sistema con journalctl

- `journalctl -u ssh`
- `journalctl -b` # desde el arranque
- `journalctl -f` # en tiempo real
- `journalctl --since "1 hour ago"`

## 30 EJERCICIOS PRÁCTICOS DEL MÓDULO 5

Divididos en básicos, intermedios, avanzados y proyecto final.

---

### EJERCICIOS BÁSICOS (1–10)

#### 1. Mostrar todos los procesos activos

- `ps aux`
- 

#### 2. Mostrar procesos en formato árbol

- pstree
- 

### 3. Ver los 10 procesos que más CPU utilizan

- top

Usa P dentro de top.

---

### 4. Ver los procesos que más memoria consumen

- top

Usa M.

---

### 5. Filtrar procesos por usuario

- ps -u \$USER
- 

### 6. Encontrar el PID de un programa específico: systemd

- pidof systemd

### 7. Encontrar el PID de un programa específico: bash

- pidof bash

#### nota

otra instrucción que podemos usar es:

ps aux | grep firefox

---

### 8. Matar un proceso por PID

- kill PID
- 

### 9. Matar procesos por nombre

- pkill firefox
- 

### 10. Ejecutar un comando con prioridad modificada

- `nice -n 5 sleep 200`
- 

## ✓ EJERCICIOS INTERMEDIOS (11–20)

### 11. Cambiar la prioridad de un proceso en ejecución

- `sudo renice -n 10 -p PID`
- 

### 12. Crear proceso en segundo plano

- `sleep 300 &`
- 

### 13. Traer proceso al primer plano

- `fg`
- 

### 14. Enviar proceso al segundo plano

- `Ctrl + Z`
  - `bg`
- 

### 15. Ver trabajos en background

- `jobs`
- 

### 16. Ver servicios activos

- `systemctl --type=service --state=running`
- 

### 17. Mostrar estado del servicio ssh

- `systemctl status ssh`
- 

### 18. Detener el servicio ssh

- `sudo systemctl stop ssh`
- 

### 19. Iniciar el servicio ssh

- `sudo systemctl start ssh`
- 

## 20. Ver logs del servicio ssh

- `journalctl -u ssh`
- 

## ✅ EJERCICIOS AVANZADOS (21–30)

### 21. Habilitar servicio para que arranque automáticamente

- `sudo systemctl enable ssh`
- 

### 22. Deshabilitar un servicio en el arranque

- `sudo systemctl disable bluetooth`
- 

### 23. Crear un archivo de log personalizado

Ejecuta:

- `echo "Test log" | systemd-cat -t curso-linux`

para ver el log:

- `journalctl -t curso-linux`

`sleep 30`: hace que el sistema o el script **espere 30 segundos**.

### 24. Buscar tu log personalizado

- `journalctl -t curso-linux`
- 

### 25. Ver logs desde el último reinicio

- `journalctl -b`
- 

### 26. Identificar un proceso zombie

Es un proceso que ya ha terminado de ejecutarse pero todavía aparece en el sistema como pendiente de ser eliminado

- `ps aux | grep Z`
-

## 27. Crear el servicio persistente

- `sudo nano /etc/systemd/system/hola.service`

### Contenido:

```
[Unit]

Description=Servicio de prueba persistente

[Service]

ExecStart=/bin/bash -c 'while true; do echo "Hola desde systemd: $(date)"; sleep 10; done'

Restart=always

RestartSec=2

StandardOutput=journal

StandardError=journal

[Install]

WantedBy=multi-user.target
```

### Recargar systemd

- `sudo systemctl daemon-reload`

### Iniciar / habilitar

- `sudo systemctl start hola`
- `sudo systemctl enable hola`

### Ver el log en tiempo real

- `Sudo journalctl -u hola -f`

### Pararlo (recomendado incluir)

- `sudo systemctl stop hola`
- `sudo systemctl disable hola`



## MINI-PROYECTO DEL MÓDULO 5

### Objetivo:

Crear y administrar una infraestructura básica de procesos y servicios.

### Tareas:

1. Crear un script en Bash llamado `monitor.sh` que:
  - Muestre uso de CPU

- Muestre memoria libre
  - Muestre 5 procesos con más consumo
2. Crear un servicio systemd monitor.service que ejecute el script.
  3. Hacer que el servicio arranque automáticamente:

`sudo systemctl enable monitor`

4. Verificar funcionamiento:

`systemctl status monitor`

5. Revisar logs:

`journalctl -u monitor`

6. Documentar todo lo realizado.

Posible solución:

### 1) Script simple: monitor.sh

- `sudo nano /usr/local/bin/monitor.sh`

Pega esto:

```
#!/bin/bash
```

```
echo "===== MONITOR $(date) ====="
```

```
echo
```

```
echo "CPU (top 1 linea):"
```

```
top -bn1 | head -n 5
```

```
echo
```

```
echo "Memoria libre (free -h):"
```

```
free -h
```

```
echo
```

```
echo "Top 5 procesos por CPU:"
```

```
ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head -n 6
```

```
echo "=====
```

Permisos:

- `sudo chmod +x /usr/local/bin/monitor.sh`

Prueba:

- `/usr/local/bin/monitor.sh`
- 

## 2) Servicio systemd fácil: monitor.service (se repite cada 60s)

- `sudo nano /etc/systemd/system/monitor.service`

Pega esto:

```
[Unit]
```

```
Description=Monitor basico (CPU/mem/procesos)
```

```
[Service]
```

```
Type=simple
```

```
ExecStart=/bin/bash -c 'while true; do /usr/local/bin/monitor.sh; sleep 60; done'
```

```
StandardOutput=journal
```

```
StandardError=journal
```

```
Restart=always
```

```
RestartSec=5
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Recarga systemd:

- `sudo systemctl daemon-reload`
- 

## 3) Arrancar y habilitar

- `sudo systemctl start monitor`
  - `sudo systemctl enable monitor`
- 

## 4) Verificar

- `sudo systemctl status monitor --no-pager`
- 

## 5) Ver logs

Últimas líneas:

- `sudo journalctl -u monitor -n 50 --no-pager`

En directo:

- `sudo journalctl -u monitor -f`
- 

# MÓDULO 6 — REDES EN LINUX

---

## 6.1. Comandos de red fundamentales

Linux utiliza herramientas muy potentes para la administración de red.

### ♦ **Mostrar interfaces de red**

- `ip a`

### ♦ **Ver rutas**

- `ip r`

### ♦ **Ping (conectividad)**

- `ping 8.8.8.8`

### ♦ **Resolver nombres (DNS)**

- `nslookup google.com`
- `dig google.com`

### ♦ **Mostrar conexiones activas**

- `ss -tunap`

### ♦ **Trazar rutas**



- traceroute google.com
- 

## 6.2. Configuración de red con Netplan (Ubuntu Server)

Netplan usa archivos YAML en:

- /etc/netplan/\*.yaml

Ejemplo IP estática:

network:

version: 2

ethernets:

enp0s3:

dhcp4: no

addresses: [192.168.1.50/24]

gateway4: 192.168.1.1

nameservers:

addresses: [8.8.8.8,1.1.1.1]

Aplicar cambios:

- sudo netplan apply
- 

## 6.3. Configuración con NetworkManager (Ubuntu Desktop)

NetworkManager permite:

- DHCP
- IP estática
- DNS
- Conexiones VPN
- WiFi

Herramienta CLI:

- nmcli device status
  - nmcli connection show
-

## 6.4. Puertos, sockets y conexiones activas

Ver puertos:

- `ss -tulnp`

Significado de flags:

- `t` → TCP
  - `u` → UDP
  - `l` → listening
  - `n` → no resolver nombres
  - `p` → mostrar procesos
- 

## 6.5. Servidor SSH

Instalar:

- `sudo apt install openssh-server`

Comprobar estado:

- `systemctl status ssh`

Archivo de configuración:

- `/etc/ssh/sshd_config`

Cambiar puerto:

- Port 2222

Reiniciar:

- `sudo systemctl restart ssh`
- 

## 6.6. Firewall UFW

Activar UFW:

- `sudo ufw enable`

Permitir SSH:

- `sudo ufw allow 22/tcp`

Denegar tráfico:

- `sudo ufw deny 80/tcp`

Ver reglas:

- `sudo ufw status numbered`
- 

## 6.7. Diagnóstico de fallos de red

Pruebas:

1. Ping a localhost:
2. `ping 127.0.0.1`
3. Ping a tu IP.
4. Ping al gateway.
5. Ping a un DNS (8.8.8.8).
6. Ping a un dominio.

Interpretación:

| Funcion<br>a         | Falla   | Significa                     |
|----------------------|---------|-------------------------------|
| <b>127.0.0.1</b>     | —       | fallo grave de TCP/IP         |
| <b>IP<br/>propia</b> | —       | fallo de driver o NIC         |
| <b>Gateway</b>       | —       | fallo de red local            |
| <b>8.8.8.8</b>       | —       | fallo de salida a<br>Internet |
| <b>DNS</b>           | dominio | fallo de resolución<br>DNS    |

---

## 30 EJERCICIOS PRÁCTICOS DEL MÓDULO 6

---

### BÁSICOS (1–10)

#### 1. Mostrar interfaces de red

- `ip a`

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a6:ff:78 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 84831sec preferred_lft 84831sec
    inet6 fd17:625c:f037:2:52d0:b675:dbaa:6986/64 scope global temporary dynamic
        valid_lft 85880sec preferred_lft 13880sec
    inet6 fd17:625c:f037:2:a00:27ff:fea6:ff78/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 85880sec preferred_lft 13880sec
    inet6 fe80::a00:27ff:fea6:ff78/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$
```

## 2. Mostrar la ruta por defecto

- ip r

```
(kali㉿kali)-[~]
$ ip r
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
```

## 3. Hacer ping a tu puerta de enlace

Ejemplo:

- ping 192.168.1.1

```
(kali㉿kali)-[~]
$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
```

## 4. Consultar DNS con nslookup

- nslookup google.com

```
(kali㉿kali)-[~]
$ nslookup google.com
Server:      80.58.61.254
Address:     80.58.61.254#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.200.110
Name:   google.com
Address: 2a00:1450:4006:80a::200e

(kali㉿kali)-[~]
$
```

## 5. Ver conexiones activas

- ss -tunap

```
(kali㉿kali)-[~]
$ ss -tunap
Netid    State    Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
udp      ESTAB    0          0          10.0.2.15%eth0:68      10.0.2.2:67
```

## 6. Usar traceroute

- traceroute 8.8.8.8

```
(kali㉿kali)-[~]
$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  10.0.2.2 (10.0.2.2)  0.678 ms  0.618 ms  0.782 ms
```

## 7. Instalar net-tools y usar ifconfig (opcional)

- sudo apt install net-tools

```
(kali㉿kali)-[~]
$ sudo apt install net-tools
net-tools ya está en su versión más reciente (2.10-2).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
 bloodhound.py          libfuse2t64            libpocketsphinx3       mesa-vaapi-drivers
 curlftpfs              libgav1-1              libpostproc58          pocketsphinx-en-us
 libavfilter10          libmjpegutils-2.1-0t64 libsphinxbase3t64      python3-fs
 libavformat61          libmpeg2encpp-2.1-0t64 libswscale8            vdpau-driver-all
 libconfig-inifiles-perl libmpx2-2.1-0t64       libvdpau-va-gl1
Utilice «sudo apt autoremove» para eliminarlos.

Resumen:
  Actualizando: 0, Instalando 0, Eliminando: 0, no actualizando: 9
```

- ifconfig

```
(kali㉿kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::a00:27ff:fea6:ff78 prefixlen 64 scopeid 0x20<link>
    inet6 fd17:625c:f037:2:a00:27ff:fea6:ff78 prefixlen 64 scopeid 0x0<global>
    inet6 fd17:625c:f037:2:52d0:b675:dbaa:6986 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:a6:ff:78 txqueuelen 1000 (Ethernet)
    RX packets 228500 bytes 241293079 (230.1 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 71965 bytes 86087040 (82.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

## 8. Ver tu hostname

- hostname

```
(kali㉿kali)-[~]  
$ hostname  
kali
```

## 9. Modificar temporalmente tu IP

- sudo ip a add 192.168.1.200/24 dev enp0s3

```
(kali㉿kali)-[~]  
$ sudo ip a add 192.168.1.200/24 dev eth0
```

## 10. Quitar una IP añadida

- sudo ip a del 192.168.1.200/24 dev enp0s3

```
(kali㉿kali)-[~]  
$ sudo ip a del 192.168.1.200/24 dev eth0
```

## 11. Repite el ejercicio anterior con otra ip

---

## ✓ INTERMEDIOS (12–20)

---

## 12. Aplicar la configuración

- sudo netplan apply
- 

## 13. Ver estado de NetworkManager (Desktop)

- nmcli device status

```
(kali㉿kali)-[~]  
$ nmcli device status  
DEVICE  TYPE      STATE           CONNECTION  
eth0     ethernet  conectado      Wired connection 1  
lo       loopback  connected (externally)  lo
```

---

## 14. Revisar logs del servicio de red

- journalctl -u NetworkManager

```
(kali㉿kali)-[~]
$ journalctl -u NetworkManager
ene 21 11:57:33 kali systemd[1]: Starting NetworkManager.service - Network Manager ...
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.3801] NetworkManager (version 1.>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.3801] Read config: /etc/NetworkM>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4045] manager[0x5587dc3fa4e0]: m>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4046] monitoring ifupdown state >
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4750] hostname: hostname: using >
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4751] hostname: static hostname >
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4766] dns-mgr: init: dns=default>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4910] manager[0x5587dc3fa4e0]: r>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.4912] manager[0x5587dc3fa4e0]: r>
ene 21 11:57:33 kali NetworkManager[579]: <info> [1768993053.5035] Loaded device plugin: NMBL>
```

## 15. Escanear redes WiFi disponibles

- nmcli device wifi list

## 16. Hacer un test de velocidad simple

- sudo apt install speedtest-cli

```
(kali㉿kali)-[~]
$ sudo apt install speedtest-cli
speedtest-cli ya está en su versión más reciente (2.1.3-3).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
 bloodhound.py          libfuse2t64          libpocketsphinx3     mesa-vdpau-drivers
 curlftpfs              libgav1-1            libpostproc58        pocketsphinx-en-us
 libavfilter10          libmjpegutils-2.1-0t64  libsphinxbase3t64    python3-fs
 libavformat61          libmpeg2encpp-2.1-0t64  libswscale8          vdpau-driver-all
 libconfig-inifiles-perl libmpeg2-2.1-0t64      libvdpau-va-gl1
Utilice «sudo apt autoremove» para eliminarlos.

Resumen:
  Actualizando: 0, Instalando 0, Eliminando: 0, no actualizando: 9
```

- speedtest

```
(kali㉿kali)-[~]
$ speedtest
Retrieving speedtest.net configuration ...
Testing from Telefonica de Espana Static IP (2.136.142.51) ...
Retrieving speedtest.net server list ...
Selecting best server based on ping ...
Hosted by Jazztel (Madrid) [138.34 km]: 34.937 ms
Testing download speed.....
```

## 17. Abrir puerto para un servicio

- sudo ufw allow 8080/tcp

```
(kali㉿kali)-[~]
$ sudo ufw allow 8080/tcp
Rule added
Rule added (v6)
```

---

## 18. Bloquear puerto

- `sudo ufw deny 23/tcp`

```
(kali㉿kali)-[~]  
$ sudo ufw deny 23/tcp  
Rule added  
Rule added (v6)
```

---

## 19. Ver reglas activas de firewall

- `sudo ufw status verbose`

```
(kali㉿kali)-[~]  
$ sudo ufw status verbose  
Status: active  
Logging: on (low)  
Default: deny (incoming), allow (outgoing), disabled (routed)  
New profiles: skip  
  
To Action From  
--  
8080/tcp ALLOW IN Anywhere  
23/tcp DENY IN Anywhere  
8080/tcp (v6) ALLOW IN Anywhere (v6)  
23/tcp (v6) DENY IN Anywhere (v6)
```

---

## 20. Conectarte por SSH desde otra máquina

- `ssh usuario@IP`

---

## ✓ AVANZADOS (21–30)

### 21. Cambiar el puerto por defecto de SSH

Editar:

- `sudo nano /etc/ssh/sshd_config`

```
(kali㉿kali)-[~]  
$ sudo nano /etc/ssh/sshd_config
```

### 22. Comprobar que SSH escucha en el nuevo puerto

- `ss -tulnp | grep ssh`



---

## 23. Forzar autenticación por clave pública

Modificar:

- PasswordAuthentication no

---

## 24. Probar fallo de DNS

Cambiar DNS a uno incorrecto y probar ping a dominio.

---

## 25. Analizar tráfico activo por proceso

- ss -tp

```
(kali㉿kali)-[~]  
$ ss -tp  
State      Recv-Q      Send-Q       Local Address:Port      Peer Address:Port      Process  
LISTEN     0            128          *:*                      *:*                      sshd: /usr/sbin/sshd -D  
*:*
```

## 26. trabajo en equipo: Crear una red virtual interna entre varias VMs

- Asignar IPs manualmente y probar ping entre ellas.

```

(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:a6:ff:78 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:e4:0d:e7 brd ff:ff:ff:ff:ff:ff
   inet 192.168.0.127/23 brd 192.168.1.255 scope global dynamic noprefixroute eth1
       valid_lft 43171sec preferred_lft 43171sec
   inet6 fe80::a00:27ff:fee4:de7/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$ ping 192.168.0.119
PING 192.168.0.119 (192.168.0.119) 56(84) bytes of data.
64 bytes from 192.168.0.119: icmp_seq=1 ttl=64 time=4.12 ms
64 bytes from 192.168.0.119: icmp_seq=2 ttl=64 time=5.43 ms
64 bytes from 192.168.0.119: icmp_seq=3 ttl=64 time=3.76 ms
64 bytes from 192.168.0.119: icmp_seq=4 ttl=64 time=2.77 ms
64 bytes from 192.168.0.119: icmp_seq=5 ttl=64 time=3.00 ms
64 bytes from 192.168.0.119: icmp_seq=6 ttl=64 time=1.91 ms
64 bytes from 192.168.0.119: icmp_seq=7 ttl=64 time=2.99 ms
^C
— 192.168.0.119 ping statistics —
7 packets transmitted, 7 received, 0% packet loss, time 6011ms
rtt min/avg/max/mdev = 1.906/3.426/5.432/1.050 ms

(kali㉿kali)-[~]
$

```

Posible solución

## ✓ Parte A — Preparación (VirtualBox)

### 1) Crear/usar red “Internal Network”

En **cada VM** (Kali/Parrot/Ubuntu):

1. Apaga la VM si está encendida.
2. Ve a:  
**VirtualBox → Configuración → Red**
3. En **Adaptador 1**:
  - Marca ☒ **Habilitar adaptador de red**
  - Conectado a: **Red interna (Internal Network)**
  - Nombre: escribe el mismo para todos, por ejemplo:  
**aula-interna**
  - Modo promiscuo: **Denegar**
  - Cable conectado: ☒

📌 Importantísimo: **Todas las VMs deben tener el mismo nombre de red interna** (aula-interna).

---

## ✅ Parte B — Direccionamiento IP (manual)

Vamos a usar una red privada tipo:

- Red: 192.168.50.0/24
- Máscara: 255.255.255.0

Ejemplo de reparto por grupos:

- VM1: 192.168.50.10
- VM2: 192.168.50.11
- VM3: 192.168.50.12

(El gateway no es necesario, porque la red es interna y aislada.)

---

## ✅ Parte C — Configurar IP dentro de Linux

### ♦ Opción 1 (recomendada para clase): configuración temporal con ip

En **cada VM**, abre terminal y ejecuta:

1. Ver interfaz de red:

ip a

Localiza la interfaz (normalmente será enp0s3, eth0 o similar).

2. Asignar IP (ejemplo VM1):

```
sudo ip addr add 192.168.50.10/24 dev enp0s3
```

```
sudo ip link set enp0s3 up
```

(En otra VM usa 192.168.50.11, etc.)

3. Verificar:

```
ip a show enp0s3
```

📌 Nota: Si se equivocan de interfaz, no funcionará el ping.

---

### ♦ Opción 2: configuración persistente (Ubuntu/Debian/Parrot)

En Ubuntu (netplan) sería algo así:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

Ejemplo:

network:

version: 2

renderer: networkd

ethernets:

enp0s3:

dhcp4: no

addresses:

- 192.168.50.10/24

Aplicar:

```
sudo netplan apply
```

En Kali/Parrot también pueden usar NetworkManager:

```
nmtui
```

---

## ✅ Parte D — Pruebas de conectividad

### 1) Hacer ping entre máquinas

Desde VM1:

```
ping -c 4 192.168.50.11
```

Desde VM2:

```
ping -c 4 192.168.50.10
```

✅ Si responde: red interna OK.

---

## ✅ Parte E — Evidencias (para entregar)

Cada grupo debe entregar:

1. Captura de ip a en cada VM.
2. Captura de ping funcionando entre dos máquinas.
3. Breve explicación (5 líneas):

- qué es Internal Network
- por qué no hace falta gateway
- qué rango IP han usado

## 27. Hacer port forwarding en VirtualBox

Acceder por SSH desde tu PC Host.

## 28. Crear un script de diagnóstico de red

Debe mostrar:

- IP
- Gateway
- DNS
- Conectividad a Internet

Solución

### Script: diag\_red.sh

```
#!/bin/bash
```

```
# Script sencillo de diagnóstico de red (FP - Linux)
```

```
echo "=====
```

```
echo "  DIAGNÓSTICO DE RED (FP)  "
```

```
echo "=====
```

```
echo "Equipo: $(hostname)"
```

```
echo "Fecha : $(date)"
```

```
echo
```

```
# 1) IP
```

```
echo "1) IP (dirección del equipo):"
```

```
IP=$(hostname -I | awk '{print $1}')
```

```
if [ -z "$IP" ]; then
    echo " IP: No encontrada ❌"
else
    echo " IP: $IP ✅"
fi
echo
```

## # 2) Gateway

```
echo "2) Gateway (puerta de enlace):"
GW=$(ip route | awk '/default/ {print $3}')
if [ -z "$GW" ]; then
    echo " Gateway: No hay (normal en Red Interna de VirtualBox) ⚠️"
else
    echo " Gateway: $GW ✅"
fi
echo
```

## # 3) DNS

```
echo "3) DNS (servidores de nombres):"
DNS=$(grep "nameserver" /etc/resolv.conf 2>/dev/null | awk '{print $2}')
if [ -z "$DNS" ]; then
    echo " DNS: No encontrado ❌"
else
    echo " DNS:"
    echo "$DNS" | sed 's/^/ - /'
    echo " ✅"
fi
echo
```

#### # 4) Conectividad a Internet

```
echo "4) Conectividad a Internet:"
```

```
ping -c 2 -W 1 8.8.8.8 > /dev/null 2>&1
```

```
if [ $? -eq 0 ]; then
```

```
    echo "  Internet: OK (ping a 8.8.8.8) 
```

```
else
```

```
    echo "  Internet: NO hay conexión (ping fallido) 
```

```
fi
```

```
echo
```

```
echo "Fin del diagnóstico."
```

---

#### Cómo usarlo (alumnos)

1. Crear archivo:

```
nano diag_red.sh
```

2. Dar permisos:

```
chmod +x diag_red.sh
```

3. Ejecutar:

```
./diag_red.sh
```

---

#### 29. Simular caída de red

- `sudo ip link set enp0s3 down`

Y recuperarla:

- `sudo ip link set enp0s3 up`
- 

#### 30. Capturar paquetes (Wireshark o tcpdump)

Ejemplo:

- `sudo tcpdump -i enp0s3 -c 20`
- 

## MINI-PROYECTO DEL MÓDULO 6 EN EQUIPO

### Objetivo:

Configurar **un servidor Ubuntu accesible por SSH de forma segura** y documentar todo el proceso.

---

### Tareas:

1. Asignar **IP estática** usando Netplan.
  2. Configurar **DNS manuales**.
  3. Activar SSH y cambiar puerto.
  4. Crear clave pública/privada y desactivar login por contraseña.
  5. Configurar firewall UFW:
    - permitir nuevo puerto SSH
    - denegar puertos innecesarios
  6. Crear script `diagnostico_red.sh` con:
    - IP actual
    - Gateway
    - DNS
    - Ping a 8.8.8.8
    - Ping a google.com
  7. Probar conexión remota desde otra máquina.
  8. Documentar todo:
    - Archivos modificados
    - Comandos usados
    - Problemas encontrados
    - Solución aplicada
-



## Solución posible

### Actividad en equipo: Red interna + SSH seguro entre varios puestos (VirtualBox)

#### Objetivo

Crear una red virtual interna (aislada) donde varias máquinas (Kali/Parrot/Ubuntu) puedan:

- verse por IP (ping),
  - conectarse por **SSH de forma segura**,
  - proteger SSH con claves y firewall,
  - y documentar el proceso.
- 

#### Organización por equipos (ideal 3 alumnos / 3 VMs)

Cada equipo trabaja con **3 VMs**, por ejemplo:

- VM1: Ubuntu Desktop
- VM2: Kali
- VM3: Parrot

#### Cada VM tendrá:

- IP (dinámica o manual)
  - SSH activo (servidor)
  - claves SSH para acceder a las otras
  - firewall (en Ubuntu/Parrot con UFW; en Kali se puede omitir o usar iptables)
- 

### 0) VirtualBox: Red interna común (obligatorio)

En cada VM:

#### Adaptador 1 (opcional para Internet)

- NAT

#### Adaptador 2 (obligatorio para comunicarse)

- Red interna
- Nombre: **aula-interna**

 Importante: **el nombre debe ser el mismo en todas.**

---

## 1) IP de las máquinas (dos opciones)

### ✅ Opción A (recomendada para FP): IP manual temporal (sin Netplan)

En cada VM poned una IP distinta en la interfaz de red interna (enp0s8, eth1, etc.)

#### 1.1 Identificar interfaz

ip a

#### 1.2 Asignar IP (ejemplos)

En VM1:

```
sudo ip addr add 192.168.50.10/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

VM2:

```
sudo ip addr add 192.168.50.11/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

VM3:

```
sudo ip addr add 192.168.50.12/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

📌 Esto es temporal (al reiniciar se pierde), pero para clase es perfecto.

---

## 2) Comprobar conectividad (ping entre VMs)

Desde VM1:

```
ping -c 3 192.168.50.11
```

```
ping -c 3 192.168.50.12
```

✅ Si hay respuesta, la red interna funciona.

---

## 3) Instalar y activar SSH en TODAS las VMs

**Ubuntu / Parrot:**

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

**Kali:**

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

Comprobar:

```
sudo systemctl status ssh --no-pager
```

---

## **4) SSH seguro (cambiar puerto + claves + sin contraseña)**

### **4.1 Cambiar puerto SSH (recomendado)**

En cada VM editar:

```
sudo nano /etc/ssh/sshd_config
```

Cambiar:

```
#Port 22
```

por:

```
Port 2222
```

Y añadir (o asegurar):

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

Reiniciar:

```
sudo systemctl restart ssh
```

Comprobar puerto:

```
ss -tulpn | grep ssh
```

---

### **4.2 Crear clave SSH en cada máquina (cada una tendrá su clave)**

En cada VM:

```
ssh-keygen -t ed25519
```

(Enter, Enter, Enter)

---

#### 4.3 Copiar clave a las otras máquinas

Ejemplo: desde VM1 copiar clave a VM2 y VM3:

```
ssh-copy-id -p 2222 usuario@192.168.50.11
```

```
ssh-copy-id -p 2222 usuario@192.168.50.12
```

Y desde VM2 copiar a VM1 y VM3, etc.

🔑 Resultado: cualquiera puede entrar por SSH a cualquiera **sin contraseña**.

---

#### 5) Firewall (UFW) en Ubuntu / Parrot

En Ubuntu y Parrot:

```
sudo apt install ufw -y
```

```
sudo ufw allow 2222/tcp
```

```
sudo ufw enable
```

```
sudo ufw status verbose
```

✅ Solo queda abierto el puerto 2222.

En Kali no es obligatorio usar UFW (puede hacerse con iptables, pero para FP lo puedes dejar opcional).

---

#### 6) Script diagnostico\_red.sh (en cada VM)

Crear:

```
nano diagnostico_red.sh
```

Pegar:

```
#!/bin/bash
```

```
echo "=====
```

```
echo " DIAGNÓSTICO DE RED (FP)      "
```

```
echo "=====
```

```
echo "Equipo: $(hostname)"
```

```
echo "Fecha : $(date)"
```

```
echo
```

```
echo "IP actual:"
```

```
hostname -I | awk '{print " - ", $1}'
```

```
echo
```

```
echo "Gateway:"
```

```
GW=$(ip route | awk '/default/ {print $3}')
```

```
[ -z "$GW" ] && echo " - No hay gateway (red interna)" || echo " - $GW"
```

```
echo
```

```
echo "DNS:"
```

```
grep nameserver /etc/resolv.conf | awk '{print " - ", $2}'
```

```
echo
```

```
echo "Ping a 8.8.8.8:"
```

```
ping -c 2 -W 1 8.8.8.8 >/dev/null 2>&1
```

```
[ $? -eq 0 ] && echo " - OK  " || echo " - FAIL  "
```

```
echo
```

```
echo "Ping a google.com:"
```

```
ping -c 2 -W 1 google.com >/dev/null 2>&1
```

```
[ $? -eq 0 ] && echo " - OK  " || echo " - FAIL  "
```

```
Permisos:
```

```
chmod +x diagnostico_red.sh
```

```
./diagnostico_red.sh
```

---

## 7) Pruebas finales (conexiones SSH cruzadas)

Desde VM1:

```
ssh -p 2222 usuario@192.168.50.11
```

```
ssh -p 2222 usuario@192.168.50.12
```

Desde VM2:

```
ssh -p 2222 usuario@192.168.50.10
```



Debe entrar sin password.

---

## 8) Documentación (muy sencilla para FP)

Lo que deben entregar (1 PDF por equipo)

### A) Tabla de equipos

| VM  | SO     | IP            | Puerto SSH |
|-----|--------|---------------|------------|
| VM1 | Ubuntu | 192.168.50.10 | 2222       |
| VM2 | Kali   | 192.168.50.11 | 2222       |
| VM3 | Parrot | 192.168.50.12 | 2222       |

### B) Archivos modificados

- /etc/ssh/sshd\_config

### C) Comandos usados (copiar y pegar)

- ip a, ip addr add...
- apt install openssh-server
- ssh-keygen
- ssh-copy-id
- ufw allow 2222/tcp, ufw enable

### D) Evidencias (capturas)

- ping entre VMs
- salida de ./diagnostico\_red.sh
- conexión SSH realizada

## E) Problemas encontrados + solución

Ejemplos:

- No responde ping → red interna mal configurada (nombre distinto)
- SSH no conecta → UFW no permitía 2222
- No entra por clave → no se ejecutó ssh-copy-id

## MÓDULO 7 — GESTIÓN DE PAQUETES Y REPOSITORIOS (APT, SNAP, .DEB)

### 7.1. ¿Qué es un paquete en Linux?

Un **paquete** es un archivo que contiene:

- el programa,
- sus dependencias,
- scripts de instalación y configuración.

En Ubuntu se usan tres sistemas principales:

✓ **APT (paquetes .deb) — el más usado**

✓ **Snap — autocontenidos**

✓ **Flatpak (opcional)**

---

### 7.2. APT: el gestor de paquetes de Ubuntu

Comando para actualizar la lista de paquetes:

```
sudo apt update
```

Actualizar programas instalados:

```
sudo apt upgrade
```

Instalar un programa:

```
sudo apt install nombre_paquete
```

Eliminar paquete:

```
sudo apt remove nombre_paquete
```

Eliminar su configuración:

```
sudo apt purge nombre_paquete
```

Buscar paquetes:

```
apt search nombre
```

Ver información:

```
apt show nombre
```

---

### 7.3. Limpieza del sistema APT

Eliminar dependencias no usadas:

```
sudo apt autoremove
```

Limpiar la caché de descargas:

```
sudo apt clean
```

Ver espacio ocupado por paquetes:

```
du -sh /var/cache/apt
```

---

### 7.4. Gestión de repositorios APT

Los repositorios están en:

```
/etc/apt/sources.list
```

```
/etc/apt/sources.list.d/
```

Añadir repositorio:

```
sudo add-apt-repository ppa:nombre/ppa
```

Actualizar:

```
sudo apt update
```

---



## 7.5. Paquetes .deb

Instalar un .deb descargado:

```
sudo dpkg -i archivo.deb
```

Si faltan dependencias:

```
sudo apt -f install
```

Eliminar paquete instalado desde .deb:

```
sudo dpkg -r nombre
```

---

## 7.6. Snap: paquetes autocontenidos

Listar Snaps:

```
snap list
```

Instalar:

```
sudo snap install nombre
```

Eliminar:

```
sudo snap remove nombre
```

Actualizar todos los snaps:

```
sudo snap refresh
```

---

## 7.7. Flatpak (opcional si lo deseas en FP)

Instalar:

```
sudo apt install flatpak
```

---

## 30 EJERCICIOS PRÁCTICOS DEL MÓDULO 7

---

### BÁSICOS (1–10)

#### 1. Actualizar la lista de paquetes

```
sudo apt update
```

#### 2. Instalar el paquete htop

`sudo apt install htop`

### **3. Desinstalar htop**

`sudo apt remove htop`

### **4. Buscar el paquete “curl”**

`apt search curl`

### **5. Ver información del paquete curl**

`apt show curl`

### **6. Instalar curl si no está instalado**

`sudo apt install curl`

### **7. Actualizar todos los paquetes del sistema**

`sudo apt upgrade`

### **8. Ver qué paquetes se han actualizado recientemente**

`grep "upgrade" /var/log/dpkg.log`

### **9. Ejecutar limpieza del sistema**

`sudo apt autoremove`

### **10. Limpiar caché del APT**

`sudo apt clean`

---

## **✓ INTERMEDIOS (11–20)**

### **11. Abrir el archivo de repositorios**

`sudo nano /etc/apt/sources.list`

### **12. Comprobar repositorios activos**

`ls /etc/apt/sources.list.d/`

### **13. Añadir un PPA (de prueba, recomendado uno seguro)**

Ejemplo:

`sudo add-apt-repository ppa:deadsnakes/ppa`

`sudo apt update`

### **14. Instalar una versión concreta de Python**

```
sudo apt install python3.10
```

#### **15. Revertir instalación (quitar el PPA si quieres)**

```
sudo add-apt-repository --remove ppa:deadsnakes/ppa
```

```
sudo apt update
```

#### **16. Descargar un .deb de Google Chrome**

```
wget https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb
```

#### **17. Instalar ese .deb**

```
sudo dpkg -i google-chrome*.deb
```

#### **18. Reparar dependencias si fallan**

```
sudo apt -f install
```

#### **19. Quitar el paquete instalado vía .deb**

```
sudo dpkg -r google-chrome-stable
```

#### **20. Mostrar paquetes instalados manualmente**

```
apt-mark showmanual
```

---

### **✅ AVANZADOS (21–30)**

#### **21. Listar todos los paquetes instalados**

```
dpkg -l
```

#### **22. Ver qué paquete instaló un comando**

```
dpkg -S /usr/bin/python3
```

#### **23. Bloquear actualizaciones de un paquete**

```
sudo apt-mark hold nombre_paquete
```

#### **24. Desbloquearlo**

```
sudo apt-mark unhold nombre_paquete
```

#### **25. Ver dependencias de un paquete**

```
apt depends nombre
```

#### **26. Ver qué paquetes dependen de él**

```
apt rdepends nombre
```

## 27. Ver tamaño de un paquete antes de instalarlo

apt show paquete | grep Size

## 28. Instalar snap VLC

sudo snap install vlc

## 29. Comparar versiones APT y SNAP

apt show vlc # si existe en apt

snap info vlc

## 30. Crear script automatizado de actualización

Crea actualizar.sh:

```
#!/bin/bash
```

```
sudo apt update && sudo apt upgrade -y && sudo snap refresh
```

Hazlo ejecutable:

```
chmod +x actualizar.sh
```

---

## MINI-PROYECTO DEL MÓDULO 7

### Objetivo:

Demostrar que sabes **instalar, actualizar, eliminar, diagnosticar y automatizar** paquetes en Ubuntu.

---

### Tareas del proyecto:

1. **Instalar 5 programas** usando APT  
Ejemplo: htop, git, curl, neofetch, apache2.
2. **Instalar 2 paquetes SNAP**
3. **Instalar un .deb manualmente**
4. **Generar un informe con:**
  - Nombre del paquete
  - Método de instalación (apt/snap/deb)
  - Comando usado
  - Versión instalada

- Si tiene dependencias

5. **Crear un script** llamado mantenimiento.sh que:

- Actualice APT
- Limpie cache
- Actualice Snaps
- Muestre espacio en disco

6. **Automatizar ese script con cron** (cada domingo a las 22:00):

7. `crontab -e`

8. `0 22 * * SUN /ruta/mantenimiento.sh`

9. Entregar documentación indicando:

- Problemas encontrados
  - Cómo los solucionaste
  - Capturas de pantalla (opcional)
- 

## MÓDULO 8 — SCRIPTING EN BASH (AUTOMATIZACIÓN EN LINUX)

### 8.1. ¿Qué es un script en Bash?

Un **script** es un archivo de texto que contiene comandos que el sistema ejecuta de forma automática, como si los escribieras uno a uno en la terminal.

Son esenciales para:

- Automatizar tareas repetitivas
- Realizar mantenimiento
- Monitorizar sistemas
- Gestionar backups
- Crear herramientas propias

Un script Bash empieza siempre con la línea:

```
#!/bin/bash
```

---

## ■ 8.2. Crear y ejecutar un script

1. Crear archivo:

```
nano mi_script.sh
```

2. Añadir:

```
#!/bin/bash
```

```
echo "Hola, mundo"
```

3. Hacerlo ejecutable:

```
chmod +x mi_script.sh
```

4. Ejecutarlo:

```
./mi_script.sh
```

---

## ■ 8.3. Variables en Bash

Definir variable:

```
nombre="Alberto"
```

Llamarla:

```
echo $nombre
```

Variables especiales:

- \$0 → nombre del script
  - \$1 \$2 \$3 → argumentos
  - \$# → cantidad de argumentos
  - \$@ → todos los argumentos
- 

## ■ 8.4. Entrada de usuario (read)

```
read -p "Introduce tu nombre: " nombre
```

```
echo "Hola $nombre"
```

---

## ■ 8.5. Condicionales (if)

Ejemplo:

```
if [ $edad -ge 18 ]; then
    echo "Eres mayor de edad"
else
    echo "Eres menor de edad"
fi
```

---

## 8.6. Bucles (for, while)

**Bucle for:**

```
for i in {1..5}; do
    echo "Número $i"
done
```

**Bucle while:**

```
contador=1
while [ $contador -le 5 ]; do
    echo $contador
    ((contador++))
done
```

---

## 8.7. Funciones en Bash

```
saludar() {
    echo "Hola $1"
}

saludar "alumno"
```

---

## 8.8. Control de errores

```
comando || echo "El comando falló"
```

---

## 8.9. Logs dentro de scripts

```
echo "Backup completado $(date)" >> /var/log/backup.log
```

---

## 8.10. Automatización con cron

Editar tareas programadas:

```
crontab -e
```

Formato:

```
* * * * * comando
```

Ejemplo: ejecutar script todos los días a las 22:00:

```
0 22 * * * /home/user/backup.sh
```

---

## 36 EJERCICIOS PRÁCTICOS DEL MÓDULO 8

Divididos en básicos, intermedios, avanzados y un mini-proyecto.

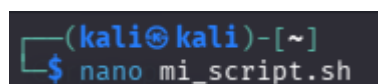
---

### ✓ EJERCICIOS BÁSICOS (1–12)

#### 1. Crear un script que muestre tu nombre

Mostrar:

```
echo "Mi nombre es ..."
```

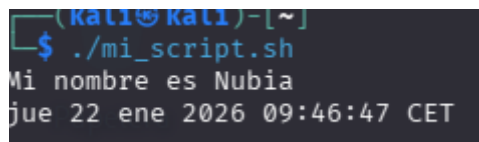


```
(kali㉿kali)-[~]  
$ nano mi_script.sh
```

---

#### 2. Script que muestre la fecha actual

```
date
```



```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 09:46:47 CET
```

---

#### 3. Script que muestre el directorio actual

```
pwd
```



```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 09:49:26 CET  
/home/kali
```

---

#### 4. Script que reciba un argumento y lo muestre

Ejemplo:

`./script.sh hola`

```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 10:08:54 CET  
/home/kali  
Introduce una palabra: hola  
Palabra introducida: hola
```

---

#### 5. Script que cuente del 1 al 10 (for)

```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 10:23:58 CET  
/home/kali  
Introduce una palabra: hola  
Palabra introducida: hola  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

---

#### 6. Script que pida al usuario su nombre y lo salude

```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 10:26:12 CET  
/home/kali  
Introduce una palabra: hola  
Palabra introducida: hola  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
Introduce tu nombre: Nubia  
Hola, Nubia
```

---

## 7. Script que muestre el contenido de /etc

```
(kali㉿kali)-[~]  
$ ./mi_script.sh  
Mi nombre es Nubia  
jue 22 ene 2026 10:30:02 CET  
/home/kali  
Introduce una palabra: hola  
Palabra introducida: hola  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
Introduce tu nombre: Nubia  
Hola, Nubia  
Contenido de /etc:  
total 1,6M  
-rw-r--r-- 1 root root 3,9K sep 14 13:53 adduser.conf  
-rw-r--r-- 1 root root 44 ene 21 11:56 adjtime  
drwxr-xr-x 2 root root 20K ene 21 12:34 alternatives  
drwxr-xr-x 8 root root 4,0K ene 21 12:27 apache2  
drwxr-xr-x 2 root root 4,0K ene 21 12:24 apparmor  
drwxr-xr-x 9 root root 12K ene 21 12:34 apparmor.d  
drwxr-xr-x 8 root root 4,0K ene 21 11:56 apt  
drwxr-xr-x 2 root root 4,0K ene 21 11:49 arp-scan
```

---

## 8. Script que muestre los 5 procesos que más CPU consumen

Usar ps aux.

```
(kali㉿kali)-[~]
$ ./script8.sh
LOS 5 PROCESOS QUE MÁS CPU CONSUMEN
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         2  0.0  0.0      0     0 ?        S    11:20   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    11:20   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   11:20   0:00 [kworker/R-rcu_gp]
root         5  0.0  0.0      0     0 ?        I<   11:20   0:00 [kworker/R-sync_wq]
root         6  0.0  0.0      0     0 ?        I<   11:20   0:00 [kworker/R-kvfree_rcu_reclaim]
```

---

## 9. Script que muestre espacio libre del disco

df -h

```
(kali㉿kali)-[~]
$ ./script9.sh
ESPACIO LIBRE DEL DISCO
S.ficheros      Tamaño Usados  Disp Uso% Montado en
udev             883M      0  883M   0% /dev
tmpfs            198M  1000K  197M   1% /run
/dev/sda1        24G    17G   5,8G  74% /
tmpfs            986M    4,0K  986M   1% /dev/shm
none             1,0M      0   1,0M   0% /run/credentials/systemd-journald.service
tmpfs            986M    8,0K  986M   1% /tmp
none             1,0M      0   1,0M   0% /run/credentials/getty@tty1.service
tmpfs            198M   104K  198M   1% /run/user/1000
```

---

## 10. Script que cree una carpeta automáticamente

```
(kali㉿kali)-[~]
$ ./script10.sh
CREAR CARPETA
La carpeta nueva_carpeta ha sido creada correctamente
```

---

## 11. Script que cree un archivo con tu nombre y la fecha actual

```
(kali㉿kali)-[~]
$ ./script11.sh
Nubia
jue 22 ene 2026 12:11:20 CET
```

---

## 12. Script que revise si un archivo existe o no

Usar if [ -f archivo ].

```
(kali@kali)-[~]
$ ls
carpeta_prueba Descargas Documentos Escritorio Imágenes mi_script.sh Música nubia.txt Plantillas Público script10.sh script11.sh script12.sh script8.sh script9.sh Vide
(kali@kali)-[~]
$ ./script12.sh
COMPROBAR SI UN ARCHIVO EXISTE
Introduce el nombre del archivo a consultar: nubia.txt
El archivo nubia.txt existe
```

---

## ✓ EJERCICIOS INTERMEDIOS (13–24)

### 13. Script que lea dos números y los sume

```
(kali@kali)-[~]
$ ./script13.sh
SUMAR DOS NÚMEROS
Ingresa el primer número: 1
Ingresa el segundo número: 2
La suma es: 3
```

---

### 14. Script que compruebe si un servicio está activo

systemctl is-active ssh

---

### 15. Script que reinicie automáticamente un servicio si está caído

---

### 16. Script que haga ping a un servidor y avise si falla

---

### 17. Script que liste todos los usuarios del sistema

cut -d: -f1 /etc/passwd

---

### 18. Script que genere un log en /var/log/local/

---

### 19. Script que busque archivos mayores de 100MB

---

### 20. Script que monitorice RAM cada 10 segundos

---

**21. Script que cree 10 archivos con nombres secuenciales**

---

**22. Script que elimine archivos temporales automáticamente**

---

**23. Script que comprima una carpeta en .tar.gz**

---

**24. Script que verifique conexión a Internet**

Hacer ping a 8.8.8.8.

---

 **EJERCICIOS AVANZADOS (25–36)**

**25. Script que haga copia de seguridad de /home**

---

**26. Script que compruebe si un puerto está abierto**

ss -tulnp | grep 22

---

**27. Script que envíe un mensaje de alerta si un disco supera el 80%**

---

**28. Script con menú interactivo (1. Info – 2. Red – 3. Salir)**

---

**29. Script que muestre los 10 archivos más grandes del sistema**

---

**30. Script que genere informes diarios de estado**

CPU, RAM, disco, red.

---

**31. Script que haga un backup incremental**

---

**32. Script que detecte procesos zombie**

ps aux | grep Z

---

**33. Script que reinicie automáticamente un servicio si falla**

---

**34. Script que registre la IP pública**

curl ifconfig.me

---

**35. Script que revise logs del sistema y filtre “error”**

---

**36. Programar tu script con cron para que se ejecute cada día**

---

---

## **MINI-PROYECTO DEL MÓDULO 8 — Sistema de Monitorización Completo**

### **Objetivo:**

Crear un sistema automático de monitorización realista en Ubuntu.

**Debe incluir un script llamado monitor.sh que:**

✓ Muestre:

- Fecha y hora
- Uso de CPU
- Uso de RAM
- 5 procesos más pesados
- Espacio libre en disco
- Estado de un servicio (apache2 o ssh)
- Conectividad a Internet

✓ Guarde un log en:

/var/log/monitor.log

✓ Cree alertas si:

- CPU > 80%
- RAM > 80%

- Disco > 80%

✓ Se ejecute automáticamente con cron cada 15 minutos:

```
*/15 * * * * /ruta/monitor.sh
```

✓ Incluya documentación técnica del proyecto:

- Código usado
  - Explicación
  - Capturas
  - Solución a errores
- 



## MÓDULO 9 — SEGURIDAD EN LINUX

Este módulo enseña a proteger Linux frente a ataques reales: fuerza bruta, puertos expuestos, configuraciones inseguras, malware de Linux, pruebas de intrusión y endurecimiento del sistema.

---



### 9.1. Principios básicos de seguridad en Linux

Seguridad en Linux se resume en 4 pilares:

✓ **Autenticación**

Usuarios, contraseñas, SSH, permisos.

✓ **Autorización**

Permisos, grupos, ACL, SELinux/AppArmor.

✓ **Auditoría**

Logs, seguimiento de accesos, herramientas como journalctl.

✓ **Hardening**

Reducir superficie de ataque:

- servicios mínimos
- puertos cerrados
- firewall activo
- actualizaciones frecuentes

---

## 9.2. Actualizaciones de seguridad

Actualizar motores críticos:

```
sudo apt update
```

```
sudo apt upgrade
```

```
sudo apt dist-upgrade
```

Ver paquetes de seguridad:

```
apt list --upgradable | grep security
```

---

## 9.3. Firewall UFW (Uncomplicated Firewall)

Activar:

```
sudo ufw enable
```

Permitir SSH:

```
sudo ufw allow 22/tcp
```

Bloquear acceso externo a un puerto:

```
sudo ufw deny 80/tcp
```

Permitir rango de IPs:

```
sudo ufw allow from 192.168.1.0/24 to any port 22
```

---

## 9.4. Endurecimiento de SSH (hardening)

Archivo de configuración:

```
/etc/ssh/sshd_config
```

Recomendaciones:

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
AllowUsers usuario1 usuario2
```

```
Port 2222
```

Reiniciar:



```
sudo systemctl restart ssh
```

---

## 9.5. Fail2ban: defensa contra fuerza bruta

Instalar:

```
sudo apt install fail2ban
```

Archivo de configuración:

```
/etc/fail2ban/jail.local
```

Ejemplo mínimo:

```
[sshd]
```

```
enabled = true
```

```
port = ssh
```

```
maxretry = 3
```

```
bantime = 10m
```

Ver estadísticas:

```
sudo fail2ban-client status
```

---

## 9.6. Permisos, ACL y Sticky Bit para seguridad

Recordatorio:

| Permis<br>o | Significad<br>o |
|-------------|-----------------|
| <b>r</b>    | lectura         |
| <b>w</b>    | escritura       |
| <b>x</b>    | ejecución       |

ACL para permisos específicos:

```
setfacl -m u:alumno:r archivo
```

Sticky Bit en carpetas públicas:

```
chmod +t /carpeta
```

---

## 9.7. Seguridad de red y puertos abiertos

Ver puertos:

```
ss -tulnp
```

Cerrar puertos innecesarios:

```
sudo ufw deny puerto
```

---

## 9.8. Logs del sistema para auditoría

Ver logs:

```
journalctl -xe
```

```
journalctl -u ssh
```

```
tail -f /var/log/auth.log
```

Buscar intentos de intrusión:

```
grep "Failed password" /var/log/auth.log
```

---

## 9.9. AppArmor (seguridad obligatoria)

Ver perfiles:

```
sudo aa-status
```

Desactivar/activar perfiles (cuidado):

```
sudo aa-disable programa
```

```
sudo aa-enforce programa
```

---

## 9.10. Seguridad del software instalado

Ver software instalado:

```
dpkg -l
```

Comprobar firmas:

```
apt-key list
```

---

## 30 EJERCICIOS PRÁCTICOS DEL MÓDULO 9

---

## ✓ BÁSICOS (1–10)

### 1. Actualizar paquetes de seguridad

`sudo apt update && sudo apt upgrade`

---

### 2. Mostrar puertos abiertos

`ss -tulnp`

---

### 3. Activar UFW

`sudo ufw enable`

---

### 4. Permitir solo conexiones SSH

`sudo ufw allow 22`

---

### 5. Bloquear puerto 23 (telnet)

`sudo ufw deny 23`

---

### 6. Ver reglas activas

`sudo ufw status`

---

### 7. Ver logs de acceso fallido

`grep "Failed password" /var/log/auth.log`

---

### 8. Cambiar root login a NO en SSH

Editar `/etc/ssh/sshd_config`.

---

### 9. Reiniciar SSH

`sudo systemctl restart ssh`

---

## 10. Revisar permisos de /etc/shadow

ls -l /etc/shadow

---

## ✓ INTERMEDIOS (11–20)

### 11. Instalar fail2ban

sudo apt install fail2ban

---

### 12. Activar jail SSH

Crear /etc/fail2ban/jail.local.

---

### 13. Ver estado de fail2ban

sudo fail2ban-client status

---

### 14. Ver intentos bloqueados

sudo fail2ban-client status sshd

---

### 15. Cambiar el puerto de SSH

Modificar Port en sshd\_config.

---

### 16. Permitir solo un rango de IP para SSH

sudo ufw allow from 192.168.1.0/24 to any port 22

---

### 17. Configurar Sticky Bit en carpeta compartida

chmod +t /carpeta\_compartida

---

### 18. Crear ACL para dar permisos específicos

setfacl -m u:profesor:rwX /datos

---

## 19. Mostrar ACL aplicadas

getfacl /datos

---

## 20. Detectar intentos sospechosos de sudo

grep "sudo" /var/log/auth.log

---

## ✅ AVANZADOS (21–30)

### 21. Crear un script que analice logs de seguridad

Debe buscar:

- failed
  - refused
  - invalid
- 

### 22. Crear un sistema de alertas por email

Usar mailutils (opcional) o escribir a un archivo.

---

### 23. Configurar límite de contraseñas en /etc/login.defs

Modificar:

- PASS\_MAX\_DAYS
  - PASS\_MIN\_DAYS
- 

### 24. Crear perfil AppArmor personalizado

(para FP puede ser solo simulado)

---

### 25. Comprobar si algún servicio escucha en puertos peligrosos

ss -tulnp | grep -E "21|23|139|445"

---

## 26. Detectar archivos con permisos 777

```
find / -perm 777 2>/dev/null
```

---

## 27. Detectar archivos sin dueño

```
find / -nouser 2>/dev/null
```

---

## 28. Crear backup del archivo sshd\_config

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

---

## 29. Crear informe de seguridad automático

Generar un informe:

- Puertos abiertos
  - Conexiones activas
  - Servicios activos
  - Permisos inseguros
  - Logs sospechosos
- 

## 30. Simular ataque de fuerza bruta y verificar fail2ban

Intentar SSH varias veces con clave incorrecta.

---

## MINI-PROYECTO DEL MÓDULO 9 — Endurecimiento Completo de Ubuntu

### Objetivo:

Aplicar un proceso profesional de seguridad real en un servidor Ubuntu.

---

### Tareas del proyecto:

1. Configurar firewall UFW correctamente:
  - permitir solo puertos necesarios
  - bloquear todo lo demás

2. Configurar SSH seguro:
    - puerto no estándar
    - desactivar root
    - desactivar contraseñas
    - permitir solo claves públicas
  3. Instalar y configurar fail2ban:
    - jail SSH activado
    - máximo 3 intentos
    - baneo de 10 minutos
  4. Revisar permisos del sistema:
    - detectar 777
    - corregir
    - aplicar Sticky Bit donde sea necesario
  5. Generar informe de seguridad:
    - puertos abiertos
    - servicios activos
    - logs sospechosos
    - usuarios con shell interactivo
    - procesos de riesgo
  6. Automatizar un script de auditoría básica con cron:
  7. 0 \* \* \* \* /ruta/seguridad.sh
- 

## MÓDULO 10 — PROYECTO FINAL DE ADMINISTRACIÓN LINUX

---

### **Objetivo del Proyecto Final**

Montar, configurar, asegurar y documentar **un servidor Ubuntu profesional**, incluyendo:

- Usuarios, grupos y permisos
- Servicios
- Red y firewall
- Seguridad y auditoría
- Automatización y scripting
- Backups
- Informes técnicos

El alumno entregará:

1. **El servidor configurado (VM o real)**
  2. **Documentación técnica profesional**
  3. **Logs y scripts desarrollados**
- 

## **ESCENARIO DE PROYECTO REALISTA**

El alumno trabaja como Administrador de Sistemas en un centro educativo o empresa.

Debe preparar un **servidor Ubuntu** que:

- Aloje usuarios del centro
  - Permita acceso remoto seguro
  - Automatice backups
  - Registre actividad
  - Limite servicios
  - Aplique políticas de seguridad
  - Proporcione carpetas compartidas por departamentos
- 

## **TAREAS DEL PROYECTO FINAL (30 Tareas Totales)**

---

### **BLOQUE 1 — Instalación y Preparación del Sistema (5 tareas)**

#### **1. Instalar Ubuntu Server en una VM**

- 2 CPU
- 2–4GB RAM



- 40GB de almacenamiento

## **2. Actualizar completamente el sistema**

`sudo apt update && sudo apt upgrade -y`

## **3. Instalar herramientas básicas**

`sudo apt install htop net-tools curl wget git ufw openssh-server`

## **4. Configurar hostname**

`sudo hostnamectl set-hostname servidor-centro`

## **5. Documentar características del servidor**

Incluye:

- versión de Ubuntu
  - kernel
  - hardware virtual
- 

# **BLOQUE 2 — Gestión de Usuarios y Grupos (6 tareas)**

## **6. Crear grupos del centro**

- alumnos
- profesores
- administracion

## **7. Crear usuarios**

- 5 alumnos
- 2 profesores
- 1 administración

## **8. Asignar grupos correspondientes**

## **9. Crear estructura de carpetas**

`/srv/alumnos`

`/srv/profesores`

`/srv/administracion`

`/srv/comun`

## 10. Configurar permisos correctos

- alumnos: acceso limitado
- profesores: rw para profesores
- administración: acceso completo

## 11. Aplicar SGID y Sticky Bit

Carpeta comun → sticky bit

---

## BLOQUE 3 — Red y Acceso Remoto (5 tareas)

### 12. Configurar IP estática con Netplan

Con DNS propio.

### 13. Instalar y activar SSH

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

### 14. Cambiar puerto SSH

En /etc/ssh/sshd\_config.

### 15. Crear claves públicas/privadas

Prohibir contraseñas.

### 16. Documentar pruebas de acceso remoto

SSH desde otro equipo.

---

## BLOQUE 4 — Seguridad del Sistema (6 tareas)

### 17. Configurar firewall UFW

- permitir SSH
- permitir servicio elegido (apache, ftp, etc.)
- bloquear puertos no usados

### 18. Instalar y configurar fail2ban

Jail mínima para SSH.

### 19. Revisar permisos peligrosos

```
find / -perm 777 2>/dev/null
```

## 20. Revisar usuarios con shell interactivo

```
grep -E "bash|zsh" /etc/passwd
```

## 21. Endurecer SSH completamente

- desactivar root
- desactivar password login
- cambiar puerto

## 22. Documentar medidas de seguridad aplicadas

---

## BLOQUE 5 — Servicios y Automatización (5 tareas)

### 23. Instalar un servicio a elección (uno de estos):

- Apache2 o Nginx
- FTP seguro (vsftpd)
- Samba
- Servicio propio systemd
- Servidor de base de datos (MariaDB o PostgreSQL)

### 24. Crear un script de mantenimiento

Debe mostrar:

- CPU
- RAM
- Disco
- Procesos pesados

### 25. Crear un script de backup de /srv

Generar archivo tar comprimido.

### 26. Automatizar ambos scripts con cron

Uno diario, otro semanal.

### 27. Ver logs del servicio instalado

Ejemplo:

journalctl -u apache2

---

## BLOQUE 6 — Auditoría, Logs e Informes (3 tareas)

### 28. Crear script para analizar intentos fallidos de login

Buscar:

Failed password

Invalid user

Connection closed

### 29. Guardar informe en /var/log/auditoria

Automatizarlo con cron.

### 30. Entregar un informe técnico final

Debe incluir:

- Descripción del sistema
  - Redes
  - Usuarios y grupos
  - Seguridad aplicada
  - Scripts creados
  - Logs adjuntos
  - Fallos encontrados y corregidos
- 

## RESULTADO ESPERADO DEL PROYECTO

Si el alumno completa correctamente estas 30 tareas, tendrá:

- ✓ Un servidor Ubuntu totalmente funcional
- ✓ Usuarios y grupos bien configurados
- ✓ Red estable y segura
- ✓ SSH endurecido
- ✓ Firewall correctamente ajustado
- ✓ Fail2ban activo
- ✓ Servicios instalados correctamente
- ✓ Scripting real de administración
- ✓ Backups automáticos

- ✓ Auditoría del sistema
  - ✓ Documentación técnica profesional
- 

## 🎓 RÚBRICA DE EVALUACIÓN (para FP)

| Área                        | Peso | Criterios                                              |
|-----------------------------|------|--------------------------------------------------------|
| Instalación y configuración | 15%  | Sistema actualizado, hostname, herramientas básicas    |
| Usuarios y permisos         | 20%  | Estructura correcta, permisos adecuados, SGID/Sticky   |
| Red y SSH                   | 15%  | IP estática, acceso remoto seguro, claves configuradas |
| Seguridad                   | 20%  | UFW, fail2ban, hardening, logs                         |
| Scripts y automatización    | 20%  | Scripts funcionales y automatizados con cron           |
| Informe técnico             | 10%  | Claridad, estructura, capturas, explicación            |

## APENDICE

### INSTALAR GUEST ADDITION EN PARROT

#### Instalar Guest Additions en Parrot OS (desde VirtualBox)

##### 1) En VirtualBox: insertar el CD de Guest Additions

Con Parrot encendido:

➡ Menú VirtualBox (arriba)

**Devices → Insert Guest Additions CD image...**

(En español: **Dispositivos → Insertar imagen de CD de las “Guest Additions”**)

---

##### 2) En Parrot: instalar dependencias necesarias

Abre terminal y ejecuta:

```
sudo apt update
```

```
sudo apt install -y build-essential dkms linux-headers-$(uname -r)
```

---

### 3) Montar el CD (si no se monta solo)

Primero crea carpeta:

```
sudo mkdir -p /mnt/cdrom
```

Montar:

```
sudo mount /dev/cdrom /mnt/cdrom
```

---

### 4) Ejecutar el instalador

```
cd /mnt/cdrom
```

```
sudo sh VBoxLinuxAdditions.run
```

---

### 5) Reiniciar

```
sudo reboot
```

PREVIAMENTE HAY QUE QUITAR LA ISO DE GUEST ADDITION

---

### ✓ Activar copiar/pegar y arrastrar

Después del reinicio en VirtualBox:

- **Devices → Shared Clipboard → Bidirectional**
  - **Devices → Drag and Drop → Bidirectional**
- 

### ✓ Comprobar que está instalado

En Parrot:

```
lsmod | grep vbox
```

Debe salir algo tipo:

- vboxguest
- vboxvideo
- vboxsf

# netplan

## Ejercicio sencillo: Usar Netplan para poner IP estática

### Objetivo

Configurar una interfaz de red con:

- **IP estática**
  - **DNS manual**  
y comprobar conectividad con ping.
- 

### Situación (para el ejercicio)

Vas a configurar esta red:

- **IP:** 192.168.50.10
- **Máscara:** /24 (equivale a 255.255.255.0)
- **Gateway:** (opcional) 192.168.50.1
- **DNS:** 8.8.8.8 y 1.1.1.1

Si están en **Red interna de VirtualBox**, normalmente **NO hay gateway**.

Si están en NAT o red real, sí puede haber gateway.

---

### PASOS (modo alumnos FP)

#### 1) Ver interfaces de red

ip a

Localiza el nombre de la interfaz, por ejemplo:

- enp0s3
- enp0s8

 En VirtualBox suele ser:

- enp0s3 = NAT
  - enp0s8 = Red interna
- 

#### 2) Ver archivo de Netplan

ls /etc/netplan/

Saldrá uno similar a:

- 00-installer-config.yaml
  - 01-netcfg.yaml
  - 01-network-manager-all.yaml
- 

### 3) Editar el archivo con nano

Ejemplo:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

---

### 4) Escribir configuración sencilla (IP estática + DNS)

#### ✓ Caso 1: SIN gateway (Red interna VirtualBox)

network:

version: 2

renderer: networkd

ethernets:

enp0s8:

dhcp4: no

addresses:

- 192.168.50.10/24

nameservers:

addresses: [8.8.8.8, 1.1.1.1]

✓ Ideal para red interna: solo necesitas IP y DNS.

---

#### ✓ Caso 2: CON gateway (si hay router)

network:

version: 2

renderer: networkd



ethernets:

enp0s3:

dhcp4: no

addresses:

- 192.168.50.10/24

gateway4: 192.168.50.1

nameservers:

addresses: [8.8.8.8, 1.1.1.1]

---

## 5) Guardar y aplicar Netplan

Guardar en nano:

- Ctrl + O (guardar)
- Enter
- Ctrl + X (salir)

Aplicar:

sudo netplan apply

---

## 6) Comprobar que se puso la IP

ip a

Debe aparecer 192.168.50.10.

---

## 7) Pruebas de conectividad

### Ping a otra VM (si tienes otra VM)

ping -c 3 192.168.50.11

### Ping a Internet (solo si hay gateway)

ping -c 3 8.8.8.8

### Probar DNS

ping -c 3 google.com

---

 **Problema típico: “Netplan YAML incorrecto”**

 Solución: comprobar espacios e indentación.

Puedes validar sin aplicar así:

```
sudo netplan try
```

Si funciona, confirma con Enter. Si no, se revierte solo.

---

 **Ejercicio para entregar (FP)**

Los alumnos entregan:

1. El archivo Netplan (copiado):

```
cat /etc/netplan/*.yaml
```

2. Captura de:

- ip a
- ping a otra VM

3. Explicación breve (5 líneas):

- qué es netplan
- qué han configurado
- por qué se usa /24

## EJERCICIO CON EL SSH

 **Objetivo**

Que el alumnado entienda qué es SSH y lo use para:

-  conectarse a otra máquina
  -  ejecutar comandos remotamente
  -  copiar archivos (SCP)
  -  ver que es seguro (sin contraseña usando claves, opcional)
-

## Material necesario

- **VM A (Servidor SSH):** Ubuntu / Parrot / Kali
- **VM B (Cliente SSH):** Kali / Parrot / Ubuntu

 Las dos en VirtualBox con red:

- Adaptador: **Red interna** (mismo nombre, ej. aula-interna)
  - (Opcional) NAT para actualizaciones
- 

## PARTE 1 — Preparar red (5 min)

En cada VM, ver IP:

`ip a`

Anotar IPs, por ejemplo:

- VM A: 192.168.50.10
- VM B: 192.168.50.11

Comprobar conectividad:

Desde VM B:

`ping -c 3 192.168.50.10`



Si responde, la red está OK.

---

## PARTE 2 — Instalar y activar SSH en la VM A (servidor)

En VM A:

`sudo apt update`

`sudo apt install openssh-server -y`

`sudo systemctl enable ssh`

`sudo systemctl start ssh`

Comprobar:

`sudo systemctl status ssh --no-pager`

---

## PARTE 3 — Conectar por SSH desde VM B (cliente)

En VM B:

ssh usuario@192.168.50.10

Ejemplo:

ssh alumno@192.168.50.10

👉 La primera vez:

- te pedirá confirmar huella: escribe yes
- luego pide la contraseña del usuario de la VM A

✅ Si entra y cambia el prompt, está conectado.

---

## PARTE 4 — ¿Cómo sé que estoy dentro de la otra máquina?

Ejecuta dentro del SSH:

whoami

hostname

pwd

👉 Explicación para alumnos:

- whoami = usuario de la máquina remota
- hostname = nombre del equipo remoto

Para salir:

exit

---

## PARTE 5 — Ejecutar un comando remoto SIN entrar (clave de concepto)

Desde VM B:

ssh alumno@192.168.50.10 "ls -l"

Otro ejemplo:

ssh alumno@192.168.50.10 "date"

✅ Esto les enseña que SSH es **administración remota**.

---

## PARTE 6 — Copiar un archivo con SCP (muy básico)

En VM B crea un archivo:

```
echo "Hola desde el cliente" > saludo.txt
```

Cópialo a VM A:

```
scp saludo.txt alumno@192.168.50.10:/home/alumno/
```

Ahora entra por SSH y comprueba:

```
ssh alumno@192.168.50.10
```

```
cat saludo.txt
```

```
exit
```

✓ Concepto: SCP = copiar archivos por SSH.

---

## PARTE 7 (Opcional BONUS) — SSH sin contraseña con claves (muy fácil)

Esto es lo más pro y les encanta.

En VM B (cliente):

```
ssh-keygen -t ed25519
```

(Enter, Enter, Enter)

Copiar clave al servidor:

```
ssh-copy-id alumno@192.168.50.10
```

Probar:

```
ssh alumno@192.168.50.10
```

✓ Si entra sin password → perfecto.

---

## PARTE 8 — Entrega (para evaluación rápida)

Cada equipo entrega:

1. IP de cada VM y resultado de ping
2. Captura de:
  - ssh usuario@IP
  - hostname y whoami
3. Captura de copia con scp
4. (Bonus) capturas de ssh-keygen y ssh-copy-id

